**UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA**




**FACULTAD DE SISTEMAS Y TELECOMUNICACIONES**

**CARRERA DE SOFTWARE**

**MATERIA:**

INTELIGENCIA DE NEGOCIOS

**FORMATIVA:**

MI PRIMER DASHBOARD: ENTREGABLE 3

**DOCENTE:**

ING. ANTHONY ABRAHAN PACHAY ESPINOZA, MSc.

**ESTUDIANTE:**

JULIO JOSÉ DEL PEZO RODRÍGUEZ

LA LIBERTAD – ECUADOR

2026

# Contenido del documento


1. Documento final del proyecto.
2. Enlace del dataset de Kaggle.
3. Descripción y justificación del dataset.
4. Preguntas de negocio.
5. KPI definidos.
6. Descripción de limpieza y transformación.
7. Modelo dimensional.
8. Diagrama estrella.
9. Granularidad.
10. Scripts SQL de creación.
11. Scripts o proceso de carga de datos.
12. Consultas de validación.
13. Vistas y/o funciones SQL.
14. Mockup.
15. Matriz de trazabilidad.
16. Código fuente Python.
17. Proyecto Streamlit.
18. README con instrucciones de ejecución.
19. Evidencias de uso de Inteligencia Artificial.
20. **Video individual de 3 a 5 minutos.**


# 1. Documento final del proyecto

Este documento es autocontenido: reúne de forma corregida e integrada todo lo trabajado en los Entregables 1 y 2. Su numeración sigue el listado de componentes exigidos para la entrega final (ver Contenido).

Paquete del proyecto: la carpeta olist-datamart/ contiene docker-compose.yml, README.md, sql/ (scripts 01 a 05), python/, app/, docs/ y evidencia/. La carpeta archive/ (los 9 CSV, más de 100 MB) no se sube al repositorio: se descarga de Kaggle.

## 1.1 Cambios respecto a los Entregables 1 y 2

- **Entregable 1 → 2.** DIM_CLIENTE se define por customer_id (no por customer_unique_id) para que cada pedido apunte a la fila correcta; la recompra sigue usando customer_unique_id. Se agregan FACT_PAGOS, la columna es_primer_item y la bitácora dw.log_calidad.
- **Tiempo de entrega.** Pasa de 12,1 a 12,56 días: el valor anterior provenía de truncar cada entrega a días enteros (Uso 2 de la sección 19).
- **Calificación, reseñas negativas y recompra.** Cambian levemente (4,09 → 4,12; 14,7 % → 13,87 %; 3,12 % → 3,04 %) porque ahora se calculan por pedido con ítems y sin cancelados.
- **% de entregas tardías.** La definición (entrega real > fecha estimada, comparando fecha y hora) reproduce exactamente las 7.827 tardías del Entregable 1.
- **Entregable 3.** Se implementa la aplicación Streamlit sobre la capa de vistas y funciones, se integra todo en un único documento autocontenido y se vuelve a ejecutar la cadena completa de scripts con los 9 CSV para verificar los resultados.

# 2. Enlace del dataset de Kaggle

| Recurso | Detalle |
|---|---|
| Repositorio GitHub | https://github.com/julios-2005/olist-datamart |
| Dataset (Kaggle) | https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce |
| Dataset (enlace de la cátedra) | https://eva.upse.edu.ec/mod/resource/view.php?id=214723 |

# 3. Descripción y justificación del dataset

| Campo | Detalle |
|---|---|
| Nombre | Brazilian E-Commerce Public Dataset by Olist |
| Enlace de origen | https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce |
| Descripción general | Pedidos reales del marketplace brasileño Olist, que conecta pequeños comercios con los principales canales de venta. Los datos están anonimizados e incluyen pedidos, ítems, pagos, reseñas, clientes, vendedores y productos. |
| Registros y columnas | 9 archivos CSV relacionados: 1.550.922 filas y 52 columnas en total (1.000.163 filas son de geolocalización). Detalle en la tabla siguiente. |
| Período temporal | Compras del 4 de septiembre de 2016 al 17 de octubre de 2018. Los pedidos con ítems llegan hasta el 3 de septiembre de 2018; la mayor parte del volumen se concentra entre enero de 2017 y agosto de 2018. |
| Qué representa una fila | Depende de la tabla (ver abajo). La unidad analítica del proyecto es el **ítem de un pedido**. |

## 3.1 Tablas, volumen y qué representa una fila

| Archivo CSV | Filas | Cols. | Una fila representa… |
|---|---|---|---|
| olist_orders_dataset | 99.441 | 8 | Un pedido (order_id único), con estado y fechas de su ciclo de vida. |
| olist_order_items_dataset | 112.650 | 7 | Un ítem dentro de un pedido: clave (order_id, order_item_id). Un pedido con 3 unidades genera 3 filas. |
| olist_order_payments_dataset | 103.886 | 5 | Un pago parcial de un pedido: (order_id, payment_sequential). |
| olist_order_reviews_dataset | 99.224 | 7 | Una reseña asociada a un pedido (calificación 1 a 5 y comentarios). |
| olist_customers_dataset | 99.441 | 5 | Un cliente *por pedido*: customer_id cambia en cada compra; customer_unique_id identifica a la persona. |
| olist_sellers_dataset | 3.095 | 4 | Un vendedor. |
| olist_products_dataset | 32.951 | 9 | Un producto (categoría, peso y dimensiones). |
| olist_geolocation_dataset | 1.000.163 | 5 | Una coordenada asociada a un prefijo postal (con muchos duplicados). No se usa en el modelo. |
| product_category_name_translation | 71 | 2 | Una categoría de producto en portugués y su traducción al inglés. |

## 3.2 Variables principales

| Tipo | Variables (tabla) |
|---|---|
| Identificadores | order_id, customer_id, customer_unique_id, product_id, seller_id, review_id, order_item_id, payment_sequential |
| Numéricas | price, freight_value (ítems); payment_value, payment_installments (pagos); review_score (ordinal 1 a 5); peso y dimensiones del producto |
| Categóricas | product_category_name; payment_type; ciudad y estado (UF) de clientes y vendedores |
| Temporales | order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date, shipping_limit_date, fechas de la reseña |
| Estado / resultado | order_status (delivered, shipped, canceled, unavailable, invoiced, processing, created, approved); review_score como medida de satisfacción |
| Texto libre | review_comment_title, review_comment_message (no se usan en los KPI) |

## 3.3 Problemas de calidad identificados

Perfilado ejecutado con python/perfilado.py (sección 16) y confirmado con consultas sobre el esquema stg:

- **Nulos:** 2.965 pedidos sin fecha de entrega al cliente y 1.783 sin fecha de entrega al transportista (semánticos: dependen del estado); 160 sin fecha de aprobación; 610 productos sin categoría ni medidas descriptivas y 2 sin peso ni dimensiones; reseñas con título nulo (87.656) y mensaje nulo (58.247).
- **Duplicados:** 261.831 filas de geolocalización totalmente duplicadas (26 %); 814 review_id repetidos y 547 pedidos con más de una reseña (551 filas sobrantes).
- **Tipos incorrectos:** todas las fechas se leen como texto; los prefijos postales tienen ceros iniciales (p. ej. 09790) que se pierden si se leen como número.
- **Valores atípicos:** price con mediana 75, media 120,7 y máximo 6.735 (8.427 ítems, 7,5 %, sobre el límite IQR); 383 ítems con flete 0; 9 pagos con valor 0; cuotas hasta 24; 4 productos con peso 0.
- **Categorías inconsistentes:** la tabla de traducción tiene 71 categorías y products usa 73 (faltan pc_gamer y portateis_cozinha_e_preparadores_de_alimentos); payment_type incluye not_defined (3 filas); ciudades de vendedores con sufijos o caracteres inválidos.
- **Integridad:** 775 pedidos sin ítems y 1 sin pago; 8 pedidos delivered sin fecha de entrega y 6 con fecha de entrega sin estar delivered; 542 ítems pertenecen a pedidos cancelados.

### 3.4 Justificación de la selección

- Es un modelo relacional multitabla que se transforma de forma natural en un esquema en estrella.
- Cubre el ciclo completo de una venta (compra, aprobación, envío, entrega, pago y reseña), lo que permite indicadores comerciales, logísticos y de satisfacción.
- Tiene dimensiones temporales, geográficas, de producto y de vendedor con cardinalidad suficiente para un tablero interactivo.
- Presenta problemas de calidad reales que justifican una etapa de ETL.
- Es un caso conocido, con documentación previa para contrastar resultados.

### 3.5 Problema, usuarios y decisiones

**Problema o necesidad de análisis.** Un marketplace necesita saber qué vende, dónde y a quién, qué tan bien entrega y cómo esa entrega afecta la satisfacción del cliente, para enfocar categorías y regiones, corregir retrasos y mejorar la experiencia. Los datos están repartidos en nueve tablas transaccionales, sin una vista integrada que permita consultarlos con filtros.

| Usuario potencial | Información que usaría | Decisiones que podría tomar |
|---|---|---|
| Gerencia comercial | Ventas por período, categoría y región; ticket promedio | Priorizar categorías y regiones, planificar campañas y temporadas altas |
| Logística / operaciones | Tiempo de entrega, % de retrasos por estado | Renegociar transportistas, ajustar plazos estimados, detectar rutas críticas |
| Gestión de vendedores | Ventas, retrasos y calificación por vendedor; concentración | Apoyar, advertir o suspender vendedores con bajo desempeño |
| Servicio al cliente | Satisfacción y su relación con la entrega | Definir acciones de atención y retención de clientes con mala experiencia |
| Finanzas | Medios de pago y cuotas | Definir políticas de financiamiento y promociones por medio de pago |

## 4. Preguntas de negocio

| Cód. | Pregunta |
|---|---|
| P1 | ¿Cómo evolucionan las ventas en el tiempo y qué categorías de producto y regiones concentran los mayores ingresos? |
| P2 | ¿Qué tan eficiente es la logística? ¿Cuánto demora en promedio la entrega, qué porcentaje llega después de la fecha estimada y en qué estados ocurre con más frecuencia? |
| P3 | ¿Cómo se relaciona la satisfacción del cliente (review_score) con los retrasos en la entrega? |
| P4 | ¿Qué medios de pago y cuántas cuotas se usan más y cómo cambia el pago promedio según el medio? |
| P5 | ¿Qué tan concentradas están las ventas en pocos vendedores y qué proporción de clientes repite compra? |

# 5. KPI definidos

Los valores corresponden al período completo, sin filtros, y fueron obtenidos ejecutando las funciones SQL sobre la base cargada (script 04_validaciones.sql, consulta V6).

| Cód. | KPI | Fórmula sobre el modelo | Pregunta | Función SQL | Valor |
|---|---|---|---|---|---|
| K1 | Ventas totales (GMV) | SUM(precio) en pedidos no cancelados | P1 | fn_total_ventas | R$ 13.496.408,43 |

| | | | | | |
|---|---|---|---|---|---|
| K2 | Pedidos | SUM(es_primer_item) no cancelados | P1, P5 | fn_total_pedidos | 98.205 |
| K3 | Ticket promedio | GMV / pedidos | P1, P4 | fn_ticket_promedio | R$ 137,43 |
| K4 | Tiempo medio de entrega | AVG(dias_entrega), pedidos entregados con fecha | P2 | fn_tiempo_entrega | 12,56 días |
| K5 | % entregas tardías | SUM(entrega_tardia) / entregados con fecha | P2, P3 | fn_pct_entregas_tardias | 8,11 % |
| K6 | Calificación promedio | AVG(review_score) por pedido | P3 | fn_calificacion_promedio | 4,12 / 5 |
| K7 | % reseñas negativas | Pedidos con review_score ≤ 2 / pedidos con reseña | P3 | fn_pct_resenas_negativas | 13,87 % |
| K8 | Participación del flete | SUM(valor_flete) / SUM(precio) | P1, P2 | fn_participacion_flete | 16,61 % |
| K9 | % ventas del top 10 de vendedores | Ventas de los 10 mayores vendedores / ventas totales | P5 | fn_pct_top10_vendedores | 13,20 % |
| K10 | Tasa de recompra | Clientes (customer_unique_id) con más de 1 pedido / clientes | P5 | fn_tasa_recompra | 3,04 % |

Además, P4 se responde con fn_pagos_resumen() (valor, pago promedio y cuotas por medio de pago), calculado sobre FACT_PAGOS. Reglas comunes: GMV, pedidos, ticket y flete excluyen pedidos cancelados; los KPI de entrega solo usan pedidos entregados con fecha; los porcentajes están en escala 0 a 100.

## 6. Descripción de limpieza y transformación

La carga se hizo en dos capas para no perder trazabilidad: los CSV se copian tal cual al esquema stg (todas las columnas como texto) y un ETL escrito en SQL (03_etl.sql) los transforma y carga en el esquema dw (Data Mart). Así, cualquier cifra del modelo puede contrastarse con el dato original. Cada corrección, marca o rechazo queda registrado en dw.log_calidad (1.990 registros).

| **Problema** | **Tratamiento aplicado en el ETL** | **Resultado verificado** |
|---|---|---|
| Fechas como texto | Conversión a timestamp; se genera DIM_TIEMPO y las fechas del hecho se guardan como clave AAAAMMDD. | 0 errores de conversión; 1.096 días en dim_tiempo (2016 a 2018). |
| Prefijos postales sin ceros | Se cargan como texto y se rellenan a 5 caracteres (lpad). | Conserva valores como 01151 y 09790. |
| Ciudades con formato inválido | Función dw.limpiar_ciudad(): minúsculas, sin tildes, sin sufijos como "/sp" o ", brasil"; correos y números pasan a 'desconocida'. | 31 ciudades de vendedores corregidas. Los 227 clientes marcados en el Entregable 1 solo tenían apóstrofe (p. ej. *santa barbara d'oeste*) y son válidos. |
| Productos sin categoría | categoria_pt = 'sin_categoria', categoria_en = 'unknown'. | 610 productos corregidos. |
| Traducción incompleta (71 vs 73) | Se completan a mano pc_gamer y portateis_cozinha_e_preparadores_de_alimentos. | 2 categorías completadas; 0 sin traducción. |
| Peso 0 o nulo | Peso 0 pasa a NULL (dato inválido); volumen_cm3 = largo × alto × ancho (variable derivada). | 4 pesos corregidos; 6 productos sin peso. |
| Reseñas repetidas | Por pedido se conserva la más reciente (DISTINCT ON (order_id)). | 547 pedidos con más de una reseña. |
| Pagos con varios medios | En el hecho de ventas se toma el medio de mayor valor del pedido; el detalle completo va en FACT_PAGOS. | 1 pedido sin pago → 'sin_pago'. |
| Pedidos sin ítems | No tienen granularidad en FACT_VENTAS_ITEM: se excluyen y se registran. | 775 pedidos rechazados y registrados. |
| Inconsistencias de estado y fecha | No se imputan: se marcan en la bitácora y los KPI de entrega usan solo pedidos entregados con fecha. | 8 delivered sin fecha y 6 con fecha sin estar entregados, marcados. |
| Valores atípicos (precio, flete, pagos en 0) | No se eliminan (son ventas reales): se mantienen y se marcan los pagos en 0. | 9 pagos en 0 marcados. |
| Variables derivadas | valor_total_item, dias_entrega, dias_retraso, entrega_tardia, es_primer_item, region (a partir de la UF), anio_mes, es_feriado. | Ver 02_modelo_dimensional.sql y 03_etl.sql. |
| Columnas irrelevantes | Geolocalización (1.000.163 filas), comentarios de reseñas, cantidad de fotos y longitudes de nombre/descripción quedan fuera del modelo; la geografía se modela por región, estado y ciudad. | Decisión documentada desde el Entregable 1. |

### Resumen de la bitácora de calidad (dw.log_calidad)

| **Tabla de origen** | **Regla** | **Acción** | **Registros** |
|---|---|---|---|
| orders | Pedido sin ítems: no tiene granularidad en FACT_VENTAS_ITEM | rechazado | 775 |
| products | Sin categoría → 'sin_categoria' (610) y peso 0 → NULL (4) | corregido | 614 |
| order_reviews | Pedido con más de una reseña: se conserva la más reciente | corregido | 547 |
| sellers | Ciudad con formato inválido normalizada | corregido | 31 |

| | | | |
|---|---|---|---|
| orders | Estado delivered sin fecha (8) y fecha con estado distinto (6) | marcado | 14 |
| order_payments | payment_value = 0 | marcado | 9 |
| **Total** | | | **1.990** |

# 7. Modelo dimensional

**Proceso de negocio:** venta de productos a través del marketplace, con su entrega y valoración. El modelo es un esquema en estrella con una tabla de hechos principal (FACT_VENTAS_ITEM), seis dimensiones y una segunda tabla de hechos (FACT_PAGOS) que comparte cuatro de ellas.

## 7.1 Tablas, claves y medidas

| Tabla | Filas | Clave primaria | Claves foráneas / contenido |
|---|---|---|---|
| dim_tiempo | 1.096 | sk_fecha (AAAAMMDD) | fecha, año, trimestre, mes, nombre del mes, año-mes, día, día de l fin de semana, feriado |
| dim_cliente | 99.441 | sk_cliente | customer_id (único), customer_unique_id, prefijo postal, ciudad, esta región |
| dim_vendedor | 3.095 | sk_vendedor | seller_id (único), prefijo postal, ciudad, estado (UF), región |
| dim_producto | 32.951 | sk_producto | product_id (único), categoría en portugués e inglés, peso, volumen |
| dim_pago | 6 | sk_pago | tipo de pago (único): 5 tipos + sin_pago |
| dim_estado_pedido | 8 | sk_estado | order_status (único), es_entregado, es_cancelado |
| **fact_ventas_item** | 112.650 | (order_id, order_item_id) | 8 FK: sk_fecha_compra, sk_fecha_entrega, sk_fecha_estimada (3 roles dim_tiempo), sk_cliente, sk_vendedor, sk_producto, sk_pago, sk_estad |
| **fact_pagos** | 103.886 | (order_id, payment_sequential) | 4 FK: sk_fecha_compra, sk_cliente, sk_pago, sk_estado |
| log_calidad | 1.990 | id | Bitácora del ETL: tabla, clave, regla, acción |

| Medida | Tabla | Definición | Alimenta |
|---|---|---|---|
| precio | fact_ventas_item | Precio del ítem (R$), con CHECK (precio >= 0) | K1, K3, K9 |
| valor_flete | fact_ventas_item | Flete del ítem (R$), con CHECK (valor_flete >= 0) | K8 |
| valor_total_item | fact_ventas_item | precio + flete | Detalle |
| cantidad | fact_ventas_item | 1 por fila | — |
| dias_entrega | fact_ventas_item | Entrega − compra, en días con fracción (NULL si no hay entrega) | K4 |
| dias_retraso | fact_ventas_item | Entrega − fecha estimada (> 0 = tarde) | K5 |
| entrega_tardia | fact_ventas_item | 1 si la entrega real supera la estimada, 0 si no, NULL si no hay entrega | K5 |
| review_score | fact_ventas_item | Calificación 1 a 5 (CHECK), una por pedido | K6, K7 |
| es_primer_item | fact_ventas_item | 1 solo en el ítem n.º 1 de cada pedido: permite contar pedidos sin DISTINCT | K2, K3, K5 a K7 |
| valor_pago, cuotas | fact_pagos | Valor de cada pago y número de cuotas (CHECK >= 0) | P4 |

## 7.2 Decisiones de diseño

- **DIM_TIEMPO con tres roles.** El hecho tiene fechas de compra, entrega y estimada que apuntan a la misma dimensión. La fecha de entrega es nula mientras el pedido no se entrega (2.454 ítems), por eso esa clave es opcional.
- **DIM_CLIENTE a nivel de customer_id.** En Olist ese identificador cambia en cada compra; se guarda también customer_unique_id (persona real) y la recompra se mide con este último.
- **es_primer_item.** La reseña y el pedido están a nivel de pedido: un pedido de 3 ítems repetiría su reseña tres veces. Los KPI de pedido se calculan con es_primer_item = 1, sin DISTINCT.
- **Segunda tabla de hechos.** FACT_PAGOS responde P4 sin duplicar importes (un pedido puede tener varios pagos).
- **Jerarquías.** Tiempo: año > trimestre > mes > día. Geografía: región > estado > ciudad (5 regiones de Brasil derivadas de la UF con dw.region_de_uf). Producto: categoría.

# 8. Diagrama estrella

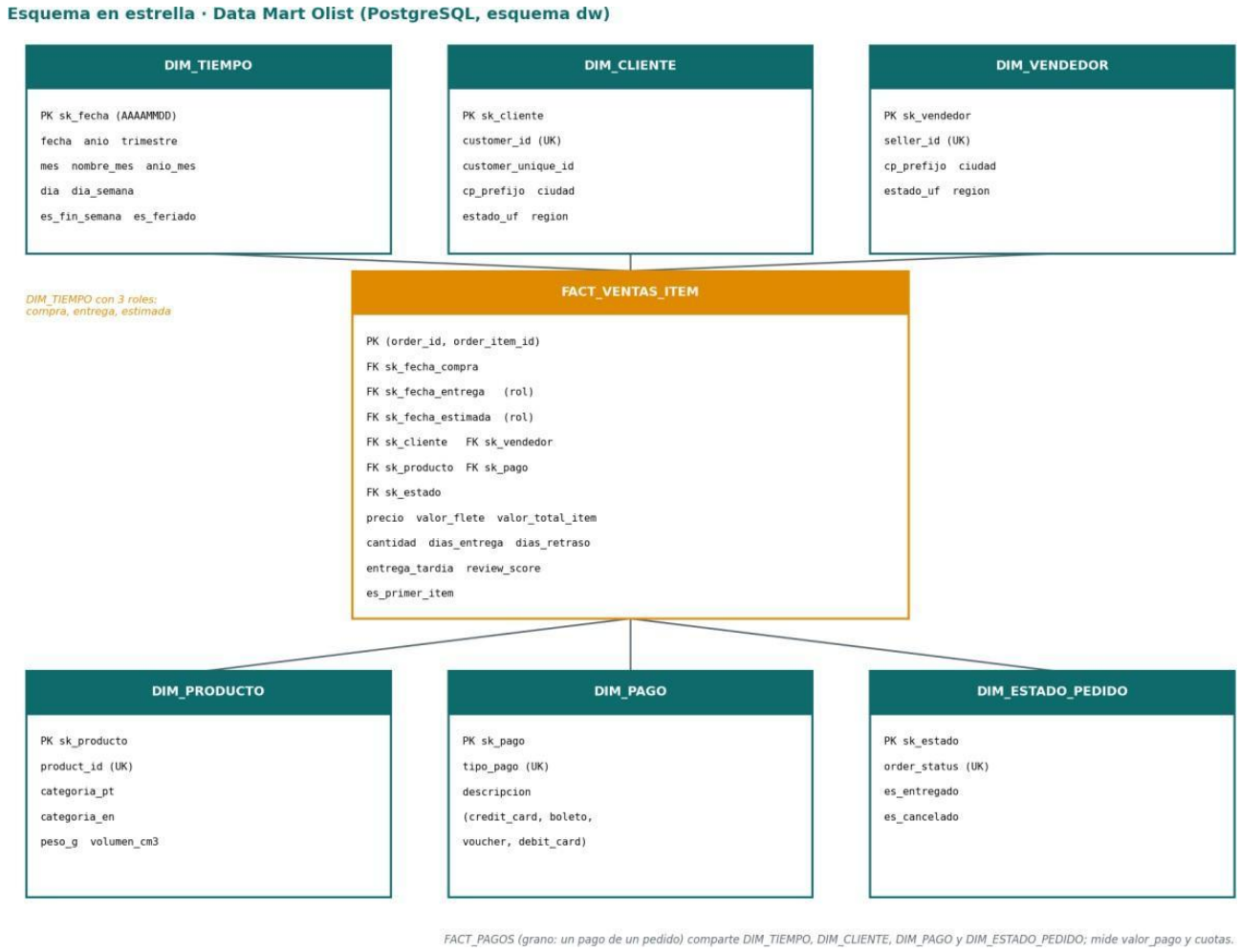



*Figura 1. Esquema en estrella del Data Mart (esquema dw).*

# 9. Granularidad

**FACT_VENTAS_ITEM:** una fila de la tabla de hechos representa un ítem (unidad vendida) dentro de un pedido, identificado por (order_id, order_item_id).

**FACT_PAGOS:** una fila de la tabla de hechos representa un pago parcial de un pedido, identificado por (order_id, payment_sequential).

# 10. Scripts SQL de creación

## 10.1 Entorno

PostgreSQL 16 se ejecuta en Docker (docker-compose.yml, sección 10.2): un entorno aislado y reproducible que se puede detener, borrar y reconstruir sin tocar el sistema operativo. La carpeta del proyecto se monta en /data (solo lectura) para que psql vea los scripts y los CSV.

| **Parámetro** | **Valor** |
|---|---|
| Host / puerto | localhost / 5432 |
| Base de datos | bi_database |
| Usuario / contraseña | bi_user / bi_password (credenciales de laboratorio) |
| Persistencia | Volumen pgdata: los datos sobreviven a docker compose down |

**10.2 docker-compose.yml**

```
# Entorno PostgreSQL para el Data Mart de Olist
# Uso:  docker compose up -d
# Conexión: host localhost | puerto 5432 | base bi_database | usuario bi_user | contraseña bi_password
services:
  db:
    image: postgres:16
    container_name: bi_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: bi_database
      POSTGRES_USER: bi_user
      POSTGRES_PASSWORD: bi_password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data   # los datos sobreviven a "docker compose down"
      - ./:/data:ro                       # esta carpeta (sql/ y archive/) visible dentro del contenedor
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bi_user -d bi_database"]
      interval: 5s
      timeout: 5s
      retries: 10

volumes:
  pgdata:
```

## 10.3 Scripts y orden de ejecución

| N.º | Script | Qué hace |
|---|---|---|
| 1 | 01_staging.sql | Crea el esquema stg y carga los 8 CSV usados con \copy, todo como texto. |
| 2 | 02_modelo_dimensional.sql | Crea el esquema dw: 6 dimensiones, 2 tablas de hechos y la bitácora, con claves primarias, foráneas, índices y restricciones CHECK. |
| 3 | 03_etl.sql | Limpieza, transformación y carga stg → dw (INSERT … SELECT); registra la bitácora. Es re-ejecutable (TRUNCATE … RESTART IDENTITY). |
| 4 | 05_vistas_funciones.sql | Crea 5 vistas y 17 funciones de consumo analítico. |
| 5 | 04_validaciones.sql | Consultas de validación (se ejecuta al final porque usa las funciones del script 05). |

```
docker compose up -d
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/01_staging.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/02_modelo_dimensional.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/03_etl.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/05_vistas_funciones.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -f /data/sql/04_validaciones.sql
```

La base bi_database la crea Docker al iniciar. -w /data/archive es necesario para que el \copy del script 01 encuentre los CSV. Para reiniciar todo el entorno: docker compose down -v.

Para este documento los cinco scripts se ejecutaron de cero, en ese orden y con ON_ERROR_STOP, sobre PostgreSQL 16 con los 9 CSV originales. Terminaron sin errores y produjeron la evidencia de las secciones 12, 13 y 17.

## 10.4 Integridad referencial y objetos creados

| Característica | Resultado |
|---|---|
| Tablas en el esquema dw | 9 (6 dimensiones, 2 hechos, 1 bitácora) |
| Claves primarias | 9 (una por tabla; compuestas en los hechos) |
| Claves foráneas | 12: 8 en fact_ventas_item y 4 en fact_pagos |

| | |
|---|---|
| Restricciones CHECK | precio ≥ 0, valor_flete ≥ 0, review_score 1 a 5, valor_pago ≥ 0, cuotas ≥ 0, trimestre y mes en rango |
| Índices | 5 en fact_ventas_item (fecha de compra, cliente, vendedor, producto, estado) y 1 en dim_cliente (customer_unique_id) |
| Vistas y funciones | 5 vistas y 17 funciones (sección 13) |

## 10.5 sql/01_staging.sql

```
-- ======================================================================
-- 01_staging.sql  |  Carga cruda de los CSV de Olist (todo como texto)
-- Ejecutar DESDE la carpeta donde están los CSV:
--   cd C:\Users\<usuario>\Downloads\archive
--   psql -U postgres -d olist_dm -f 01_staging.sql
-- Principio: staging no transforma nada; los tipos se corrigen en el ETL.
-- ======================================================================
DROP SCHEMA IF EXISTS stg CASCADE;
CREATE SCHEMA stg;

CREATE TABLE stg.customers (
   customer_id text, customer_unique_id text,
   customer_zip_code_prefix text, customer_city text, customer_state text);

CREATE TABLE stg.orders (
   order_id text, customer_id text, order_status text,
   order_purchase_timestamp text, order_approved_at text,
   order_delivered_carrier_date text, order_delivered_customer_date text,
   order_estimated_delivery_date text);

CREATE TABLE stg.order_items (
   order_id text, order_item_id text, product_id text, seller_id text,
   shipping_limit_date text, price text, freight_value text);

CREATE TABLE stg.order_payments (
   order_id text, payment_sequential text, payment_type text,
   payment_installments text, payment_value text);

CREATE TABLE stg.order_reviews (
   review_id text, order_id text, review_score text,
   review_comment_title text, review_comment_message text,
   review_creation_date text, review_answer_timestamp text);

CREATE TABLE stg.products (
   product_id text, product_category_name text, product_name_lenght text,
   product_description_lenght text, product_photos_qty text,
   product_weight_g text, product_length_cm text, product_height_cm text,
   product_width_cm text);

CREATE TABLE stg.sellers (
   seller_id text, seller_zip_code_prefix text, seller_city text, seller_state text);

CREATE TABLE stg.category_translation (
   product_category_name text, product_category_name_english text);

-- Carga (\copy es de psql y lee el archivo desde TU computador)
\copy stg.customers         FROM 'olist_customers_dataset.csv'         WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.orders            FROM 'olist_orders_dataset.csv'            WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.order_items       FROM 'olist_order_items_dataset.csv'       WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.order_payments    FROM 'olist_order_payments_dataset.csv'    WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.order_reviews     FROM 'olist_order_reviews_dataset.csv'     WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.products          FROM 'olist_products_dataset.csv'          WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
\copy stg.sellers           FROM 'olist_sellers_dataset.csv'           WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')
-- El CSV de traducción trae BOM al inicio; se carga sin HEADER y se descarta la fila 1.
\copy stg.category_translation FROM 'product_category_name_translation.csv' WITH (FORMAT csv, HEADER false, ENCODING 'UTF8')
DELETE FROM stg.category_translation WHERE product_category_name LIKE '%product_category_name';

-- La geolocalización (1.000.163 filas) no entra en la versión 1 del modelo
-- (decisión (c) del Entregable 1: la geografía se modela por estado y ciudad).

SELECT 'customers' t, count(*) FROM stg.customers UNION ALL
SELECT 'orders', count(*) FROM stg.orders UNION ALL
SELECT 'order_items', count(*) FROM stg.order_items UNION ALL
SELECT 'order_payments', count(*) FROM stg.order_payments UNION ALL
SELECT 'order_reviews', count(*) FROM stg.order_reviews UNION ALL
SELECT 'products', count(*) FROM stg.products UNION ALL
SELECT 'sellers', count(*) FROM stg.sellers UNION ALL
```

```
SELECT 'category_translation', count(*) FROM stg.category_translation;
```

## 10.6 sql/02_modelo_dimensional.sql

```
-- ============================================================================
-- 02_modelo_dimensional.sql  |  Data Mart Olist (esquema en estrella)
-- GRANULARIDAD de FACT_VENTAS_ITEM:
--   "Una fila representa un ítem (unidad vendida) dentro de un pedido:
--    clave natural (order_id, order_item_id)."
-- GRANULARIDAD de FACT_PAGOS:
--   "Una fila representa un pago parcial de un pedido:
--    clave natural (order_id, payment_sequential)."
-- ============================================================================
DROP SCHEMA IF EXISTS dw CASCADE;
CREATE SCHEMA dw;

-- ---------------------------------------------------------------------- DIMENSIONES
CREATE TABLE dw.dim_tiempo (
   sk_fecha       integer  PRIMARY KEY,        -- formato AAAAMMDD
   fecha          date     NOT NULL UNIQUE,
   anio           smallint NOT NULL,
   trimestre      smallint NOT NULL CHECK (trimestre BETWEEN 1 AND 4),
   mes            smallint NOT NULL CHECK (mes BETWEEN 1 AND 12),
   nombre_mes     text     NOT NULL,
   anio_mes       char(7)  NOT NULL,           -- 'AAAA-MM' (ordena bien en gráficos)
   dia            smallint NOT NULL,
   dia_semana     smallint NOT NULL,           -- 1 = lunes ... 7 = domingo
   nombre_dia     text     NOT NULL,
   es_fin_semana  boolean  NOT NULL,
   es_feriado     boolean  NOT NULL
);

CREATE TABLE dw.dim_cliente (
   sk_cliente         serial   PRIMARY KEY,
   customer_id        text     NOT NULL UNIQUE,  -- cambia en cada pedido
   customer_unique_id text     NOT NULL,        -- persona real (para recompra)
   cp_prefijo         char(5),                 -- texto: conserva ceros iniciales
   ciudad             text     NOT NULL,
   estado_uf          char(2)  NOT NULL,
   region             text     NOT NULL
);
CREATE INDEX ix_dim_cliente_unique ON dw.dim_cliente (customer_unique_id);

CREATE TABLE dw.dim_vendedor (
   sk_vendedor   serial   PRIMARY KEY,
   seller_id     text     NOT NULL UNIQUE,
   cp_prefijo    char(5),
   ciudad        text     NOT NULL,
   estado_uf     char(2)  NOT NULL,
   region        text     NOT NULL
);

CREATE TABLE dw.dim_producto (
   sk_producto     serial  PRIMARY KEY,
   product_id      text    NOT NULL UNIQUE,
   categoria_pt    text    NOT NULL,
   categoria_en    text    NOT NULL,
   peso_g          numeric(10,2),
   volumen_cm3     numeric(14,2)
);

CREATE TABLE dw.dim_pago (
   sk_pago       serial  PRIMARY KEY,
   tipo_pago     text    NOT NULL UNIQUE,       -- credit_card, boleto, ...
   descripcion   text    NOT NULL
);

CREATE TABLE dw.dim_estado_pedido (
   sk_estado     serial  PRIMARY KEY,
   order_status  text    NOT NULL UNIQUE,
   es_entregado  boolean NOT NULL,
   es_cancelado  boolean NOT NULL
);

-- ---------------------------------------------------------------------- TABLAS DE HECHOS
CREATE TABLE dw.fact_ventas_item (
```

```
  order_id          text     NOT NULL,
  order_item_id     smallint NOT NULL,
  -- claves foráneas (dimensiones)
  sk_fecha_compra    integer  NOT NULL REFERENCES dw.dim_tiempo (sk_fecha),
  sk_fecha_entrega   integer          REFERENCES dw.dim_tiempo (sk_fecha),  -- rol 2 (NULL si no entregado)
  sk_fecha_estimada  integer  NOT NULL REFERENCES dw.dim_tiempo (sk_fecha),  -- rol 3
  sk_cliente         integer  NOT NULL REFERENCES dw.dim_cliente (sk_cliente),
  sk_vendedor        integer  NOT NULL REFERENCES dw.dim_vendedor (sk_vendedor),
  sk_producto        integer  NOT NULL REFERENCES dw.dim_producto (sk_producto),
  sk_pago            integer  NOT NULL REFERENCES dw.dim_pago (sk_pago),    -- medio de pago principal del pedido
  sk_estado          integer  NOT NULL REFERENCES dw.dim_estado_pedido (sk_estado),
  -- medidas
  precio            numeric(10,2) NOT NULL CHECK (precio >= 0),
  valor_flete       numeric(10,2) NOT NULL CHECK (valor_flete >= 0),
  valor_total_item  numeric(10,2) NOT NULL,              -- precio + flete
  cantidad          smallint      NOT NULL DEFAULT 1,
  dias_entrega      numeric(8,2),                        -- entrega - compra (NULL si no hay entrega)
  dias_retraso      numeric(8,2),                        -- entrega - estimada (>0 = tarde)
  entrega_tardia    smallint,                            -- 1 / 0 (NULL si no hay entrega)
  review_score      smallint CHECK (review_score BETWEEN 1 AND 5),
  es_primer_item    smallint      NOT NULL,              -- 1 en el ítem #1 de cada pedido: permite contar pedidos sin DISTINCT
  PRIMARY KEY (order_id, order_item_id)
);
CREATE INDEX ix_fvi_fcompra   ON dw.fact_ventas_item (sk_fecha_compra);
CREATE INDEX ix_fvi_cliente   ON dw.fact_ventas_item (sk_cliente);
CREATE INDEX ix_fvi_vendedor  ON dw.fact_ventas_item (sk_vendedor);
CREATE INDEX ix_fvi_producto  ON dw.fact_ventas_item (sk_producto);
CREATE INDEX ix_fvi_estado    ON dw.fact_ventas_item (sk_estado);

CREATE TABLE dw.fact_pagos (
  order_id          text     NOT NULL,
  payment_sequential  smallint NOT NULL,
  sk_fecha_compra    integer  NOT NULL REFERENCES dw.dim_tiempo (sk_fecha),
  sk_cliente         integer  NOT NULL REFERENCES dw.dim_cliente (sk_cliente),
  sk_pago            integer  NOT NULL REFERENCES dw.dim_pago (sk_pago),
  sk_estado          integer  NOT NULL REFERENCES dw.dim_estado_pedido (sk_estado),
  valor_pago         numeric(10,2) NOT NULL CHECK (valor_pago >= 0),
  cuotas            smallint      NOT NULL CHECK (cuotas >= 0),
  PRIMARY KEY (order_id, payment_sequential)
);

-- Bitácora de registros descartados o corregidos por el ETL
CREATE TABLE dw.log_calidad (
  id         serial PRIMARY KEY,
  tabla      text NOT NULL,
  clave      text,
  regla      text NOT NULL,
  accion     text NOT NULL,       -- 'rechazado' | 'corregido' | 'marcado'
  registrado  timestamp NOT NULL DEFAULT now()
);
```

# 11. Scripts o proceso de carga de datos

La carga se realiza en dos pasos. Primero, 01_staging.sql copia los 8 CSV usados al esquema stg con \copy de psql, sin transformar nada (todas las columnas como texto). Después, 03_etl.sql, un ETL escrito en SQL, limpia, transforma y carga el Data Mart (esquema dw) con INSERT … SELECT: primero las dimensiones y al final las tablas de hechos, registrando cada corrección en dw.log_calidad.

El script es re-ejecutable porque vacía las tablas con TRUNCATE … RESTART IDENTITY antes de cargar. El orden de ejecución está en la sección 10.3; las reglas de limpieza, en la sección 6; y la comprobación de que la carga es correcta, en la sección 12.

## 11.1 sql/03_etl.sql

```
-- ============================================================================
-- 03_etl.sql  |  Limpieza, transformación y carga (stg -> dw)
-- Es re-ejecutable: vacía las tablas del dw antes de cargar.
-- ============================================================================
TRUNCATE dw.fact_pagos, dw.fact_ventas_item, dw.dim_tiempo, dw.dim_cliente,
       dw.dim_vendedor, dw.dim_producto, dw.dim_pago, dw.dim_estado_pedido,
       dw.log_calidad RESTART IDENTITY CASCADE;

-- ------------------------------------------------------------ funciones auxiliares
```

```
-- Normaliza texto de ciudad: minúsculas, sin tildes, sin sufijos "/sp", '', brasil", etc.
CREATE OR REPLACE FUNCTION dw.limpiar_ciudad(p text) RETURNS text
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE
    WHEN c IS NULL OR c = '' OR c ~ '@' OR c !~ '[a-z]' OR length(c) <= 2 THEN 'desconocida'
    WHEN c = 'sbc' THEN 'sao bernardo do campo'
    ELSE c
  END
  FROM (
    SELECT btrim(regexp_replace(
          split_part(split_part(split_part(split_part(
            translate(lower(coalesce(p,'')),
                  'áàâãäéèêëíìîïóòôõöúùûüçñ´'',
                  'aaaaaeeeeiiiioooooauuuucn''''' ),
          '/',1), ',',1), '\',1), '(',1),
        '\s+',' ','g')) AS c
  ) t
$$;

CREATE OR REPLACE FUNCTION dw.region_de_uf(uf text) RETURNS text
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE
    WHEN uf IN ('AC','AP','AM','PA','RO','RR','TO')           THEN 'Norte'
    WHEN uf IN ('AL','BA','CE','MA','PB','PE','PI','RN','SE')  THEN 'Nordeste'
    WHEN uf IN ('DF','GO','MT','MS')                          THEN 'Centro-Oeste'
    WHEN uf IN ('ES','MG','RJ','SP')                          THEN 'Sudeste'
    WHEN uf IN ('PR','RS','SC')                               THEN 'Sul'
    ELSE 'Desconocida' END
$$;

-- ---------------------------------------------------------------------- DIM_TIEMPO
INSERT INTO dw.dim_tiempo
SELECT to_char(d,'YYYYMMDD')::int, d::date,
    extract(year FROM d), extract(quarter FROM d), extract(month FROM d),
    (ARRAY['enero','febrero','marzo','abril','mayo','junio','julio','agosto',
        'septiembre','octubre','noviembre','diciembre'])[extract(month FROM d)::int],
    to_char(d,'YYYY-MM'),
    extract(day FROM d),
    extract(isodow FROM d),
    (ARRAY['lunes','martes','miercoles','jueves','viernes','sabado','domingo'])[extract(isodow FROM d)::int],
    extract(isodow FROM d) >= 6,
    -- Feriados nacionales de Brasil: fijos + móviles (Carnaval, Viernes Santo, Corpus Christi) 2016-2018
    to_char(d,'MM-DD') IN ('01-01','04-21','05-01','09-07','10-12','11-02','11-15','12-25')
    OR d::date IN ('2016-02-09','2016-03-25','2016-05-26',
                '2017-02-28','2017-04-14','2017-06-15',
                '2018-02-13','2018-03-30','2018-05-31')
FROM generate_series('2016-01-01'::date, '2018-12-31'::date, interval '1 day') AS g(d);

-- ---------------------------------------------------------------------- DIM_ESTADO_PEDIDO
INSERT INTO dw.dim_estado_pedido (order_status, es_entregado, es_cancelado)
SELECT DISTINCT order_status, order_status = 'delivered', order_status = 'canceled'
FROM stg.orders ORDER BY 1;

-- ---------------------------------------------------------------------- DIM_PAGO
INSERT INTO dw.dim_pago (tipo_pago, descripcion) VALUES
 ('credit_card','Tarjeta de crédito'),
 ('boleto','Boleto bancario'),
 ('voucher','Voucher / vale'),
 ('debit_card','Tarjeta de débito'),
 ('not_defined','No definido'),
 ('sin_pago','Pedido sin registro de pago');

-- ---------------------------------------------------------------------- DIM_CLIENTE
-- Una fila por customer_id (en Olist cambia en cada pedido).
-- customer_unique_id identifica a la persona real y se usa para la recompra.
INSERT INTO dw.dim_cliente (customer_id, customer_unique_id, cp_prefijo, ciudad, estado_uf, region)
SELECT customer_id, customer_unique_id,
    lpad(customer_zip_code_prefix, 5, '0'),
    dw.limpiar_ciudad(customer_city),
    upper(customer_state),
    dw.region_de_uf(upper(customer_state))
FROM stg.customers;

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'customers', customer_id, 'ciudad normalizada (minúsculas/sin tildes/sin sufijos)', 'corregido'
FROM stg.customers WHERE dw.limpiar_ciudad(customer_city) <> customer_city;

-- ---------------------------------------------------------------------- DIM_VENDEDOR
```

```
INSERT INTO dw.dim_vendedor (seller_id, cp_prefijo, ciudad, estado_uf, region)
SELECT seller_id, lpad(seller_zip_code_prefix,5,'0'),
    dw.limpiar_ciudad(seller_city), upper(seller_state), dw.region_de_uf(upper(seller_state))
FROM stg.sellers;

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'sellers', seller_id || ' | "' || seller_city || '" -> "' || dw.limpiar_ciudad(seller_city) || '"', 'ciudad con formato inválido normalizada', 'corregido'
FROM stg.sellers WHERE dw.limpiar_ciudad(seller_city) <> seller_city;

-- ---------------------------------------------------------------- DIM_PRODUCTO
-- Traducción: 71 categorías en la tabla vs 73 en products -> se completan 2 a mano.
INSERT INTO dw.dim_producto (product_id, categoria_pt, categoria_en, peso_g, volumen_cm3)
SELECT p.product_id,
    COALESCE(p.product_category_name, 'sin_categoria'),
    CASE
     WHEN p.product_category_name IS NULL THEN 'unknown'
     WHEN t.product_category_name_english IS NOT NULL THEN t.product_category_name_english
     WHEN p.product_category_name = 'pc_gamer' THEN 'pc_gamer'
     WHEN p.product_category_name = 'portateis_cozinha_e_preparadores_de_alimentos' THEN 'portable_kitchen_food_preparers'
     ELSE p.product_category_name
    END,
    NULLIF(NULLIF(p.product_weight_g,'')::numeric, 0),              -- peso 0 = dato inválido -> NULL
    NULLIF(p.product_length_cm,'')::numeric * NULLIF(p.product_height_cm,'')::numeric * NULLIF(p.product_width_cm,'')::numeric
FROM stg.products p
LEFT JOIN stg.category_translation t USING (product_category_name);

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'products', product_id, 'sin categoría -> sin_categoria', 'corregido'
FROM stg.products WHERE product_category_name IS NULL;
INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'products', product_id, 'peso = 0 -> NULL', 'corregido'
FROM stg.products WHERE product_weight_g = '0';

-- ---------------------------------------------------------------- FACT_VENTAS_ITEM
WITH pago_principal AS (   -- medio de pago de mayor valor en el pedido
  SELECT DISTINCT ON (order_id) order_id, payment_type
  FROM stg.order_payments
  ORDER BY order_id, payment_value::numeric DESC, payment_sequential::int ASC
), resena AS (             -- una reseña por pedido: la más reciente
  SELECT DISTINCT ON (order_id) order_id, review_score::smallint AS review_score
  FROM stg.order_reviews
  ORDER BY order_id, review_answer_timestamp DESC, review_creation_date DESC, review_id
), base AS (
  SELECT i.order_id,
      i.order_item_id::smallint                    AS order_item_id,
      row_number() OVER (PARTITION BY i.order_id ORDER BY i.order_item_id::int) AS rn,
      o.order_purchase_timestamp::timestamp        AS t_compra,
      o.order_delivered_customer_date::timestamp   AS t_entrega,
      o.order_estimated_delivery_date::timestamp   AS t_estimada,
      c.sk_cliente, v.sk_vendedor, p.sk_producto,
      pg.sk_pago, e.sk_estado,
      i.price::numeric AS precio, i.freight_value::numeric AS flete,
      r.review_score
  FROM stg.order_items i
  JOIN stg.orders o          ON o.order_id = i.order_id
  JOIN dw.dim_cliente c      ON c.customer_id = o.customer_id
  JOIN dw.dim_vendedor v     ON v.seller_id = i.seller_id
  JOIN dw.dim_producto p     ON p.product_id = i.product_id
  JOIN dw.dim_estado_pedido e ON e.order_status = o.order_status
  LEFT JOIN pago_principal pp ON pp.order_id = i.order_id
  JOIN dw.dim_pago pg        ON pg.tipo_pago = COALESCE(pp.payment_type, 'sin_pago')
  LEFT JOIN resena r         ON r.order_id = i.order_id
)
INSERT INTO dw.fact_ventas_item
SELECT order_id, order_item_id,
    to_char(t_compra,'YYYYMMDD')::int,
    to_char(t_entrega,'YYYYMMDD')::int,
    to_char(t_estimada,'YYYYMMDD')::int,
    sk_cliente, sk_vendedor, sk_producto, sk_pago, sk_estado,
    precio, flete, precio + flete, 1,
    round((extract(epoch FROM t_entrega - t_compra)  / 86400)::numeric, 2),
    round((extract(epoch FROM t_entrega - t_estimada) / 86400)::numeric, 2),
    CASE WHEN t_entrega IS NULL THEN NULL WHEN t_entrega > t_estimada THEN 1 ELSE 0 END,
    review_score,
    CASE WHEN rn = 1 THEN 1 ELSE 0 END
FROM base;

-- ---------------------------------------------------------------- FACT_PAGOS
```

```
INSERT INTO dw.fact_pagos
SELECT pa.order_id, pa.payment_sequential::smallint,
    to_char(o.order_purchase_timestamp::timestamp,'YYYYMMDD')::int,
    c.sk_cliente, pg.sk_pago, e.sk_estado,
    pa.payment_value::numeric, pa.payment_installments::smallint
FROM stg.order_payments pa
JOIN stg.orders o          ON o.order_id = pa.order_id
JOIN dw.dim_cliente c      ON c.customer_id = o.customer_id
JOIN dw.dim_pago pg        ON pg.tipo_pago = pa.payment_type
JOIN dw.dim_estado_pedido e  ON e.order_status = o.order_status;

-- ---------------------------------------------------- bitácora de calidad (hechos)
INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'orders', o.order_id, 'pedido sin ítems: no tiene granularidad en FACT_VENTAS_ITEM', 'rechazado'
FROM stg.orders o WHERE NOT EXISTS (SELECT 1 FROM stg.order_items i WHERE i.order_id = o.order_id);

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'orders', order_id, 'estado delivered sin fecha de entrega al cliente', 'marcado'
FROM stg.orders WHERE order_status = 'delivered' AND order_delivered_customer_date IS NULL;

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'orders', order_id, 'fecha de entrega con estado distinto de delivered', 'marcado'
FROM stg.orders WHERE order_status <> 'delivered' AND order_delivered_customer_date IS NOT NULL;

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'order_reviews', order_id, 'pedido con más de una reseña: se conserva la más reciente', 'corregido'
FROM stg.order_reviews GROUP BY order_id HAVING count(*) > 1;

INSERT INTO dw.log_calidad (tabla, clave, regla, accion)
SELECT 'order_payments', order_id || '-' || payment_sequential, 'payment_value = 0', 'marcado'
FROM stg.order_payments WHERE payment_value::numeric = 0;

ANALYZE;
SELECT regla, accion, count(*) FROM dw.log_calidad GROUP BY 1,2 ORDER BY 3 DESC;
```

# 12. Consultas de validación

Todas las cifras de esta sección salen de ejecutar 04_validaciones.sql sobre la base cargada; la salida completa está en la sección 12.6.

## 12.1 Cantidad de registros (origen stg vs destino dw)

| Tabla | Origen | Destino | Diferencia |
|---|---|---|---|
| fact_ventas_item | 112.650 | 112.650 | **0** |
| fact_pagos | 103.886 | 103.886 | **0** |
| dim_cliente | 99.441 | 99.441 | **0** |
| dim_vendedor | 3.095 | 3.095 | **0** |
| dim_producto | 32.951 | 32.951 | **0** |
| dim_estado_pedido | 8 | 8 | **0** |
| dim_pago (5 tipos + sin_pago) | 6 | 6 | **0** |
| dim_tiempo (2016 a 2018) | 1.096 | 1.096 | **0** |

Los conteos de staging coinciden con el perfilado del Entregable 1 (99.441 pedidos, 112.650 ítems, 103.886 pagos, 99.224 reseñas, 32.951 productos, 3.095 vendedores).

## 12.2 Integridad de claves

| Prueba | Huérfanos (esperado 0) |
|---|---|
| Ventas sin cliente / sin vendedor / sin producto | **0 / 0 / 0** |
| Ventas sin fecha de compra | **0** |
| Ventas con fecha de entrega inexistente en dim_tiempo | **0** |
| Pagos sin cliente | **0** |
| Pedidos con ítems en staging y sin fila en el hecho | **0** |
| Duplicados en (order_id, order_item_id) | **0** |
| Pedidos con más de un ítem marcado como primero (es_primer_item) | **0** |

Además de estas pruebas, PostgreSQL obliga a cumplir las 12 claves foráneas en cada inserción.

## 12.3 Valores nulos relevantes

| Medida | Nulos | Explicación |
|---|---|---|
| sk_fecha_entrega / dias_entrega | 2.454 | 542 ítems de pedidos cancelados o no disponibles, 1.904 aún en proceso y 8 delivered sin fecha (marcados). No se imputan. |
| review_score | 942 | Pedidos sin reseña. No se imputan. |
| precio / valor_flete | 0 | Medidas siempre presentes. |
| Productos sin categoría / sin peso | 610 / 6 | Corregidos a sin_categoria y peso NULL. |

## 12.4 Totales principales y coherencia con los datos originales

| Medida | Origen (stg) | Data Mart (dw) |
|---|---|---|
| GMV: SUM(price) sin cancelados | R$ 13.496.408,43 | **R$ 13.496.408,43** |
| Flete total | R$ 2.251.909,54 | **R$ 2.251.909,54** |
| Valor de pagos | R$ 16.008.872,12 | **R$ 16.008.872,12** |
| Pedidos con ítems | 98.666 | **98.666** |

Las cuatro medidas coinciden al centavo, lo que confirma que el ETL no pierde ni duplica importes. La diferencia entre 98.666 pedidos con ítems y 98.205 pedidos no cancelados (K2) son 461 pedidos cancelados que sí tienen ítems.

## 12.5 sql/04_validaciones.sql

```
-- ======================================================================
-- 04_validaciones.sql  |  Consultas de validación del Data Mart
-- ======================================================================
\echo '=== V1. Cantidad de registros: origen (stg) vs destino (dw) ==='
SELECT 'fact_ventas_item' AS tabla,
    (SELECT count(*) FROM stg.order_items) AS origen,
    (SELECT count(*) FROM dw.fact_ventas_item) AS destino,
    (SELECT count(*) FROM stg.order_items) - (SELECT count(*) FROM dw.fact_ventas_item) AS diferencia
UNION ALL SELECT 'fact_pagos', (SELECT count(*) FROM stg.order_payments), (SELECT count(*) FROM dw.fact_pagos), 0
UNION ALL SELECT 'dim_cliente', (SELECT count(*) FROM stg.customers), (SELECT count(*) FROM dw.dim_cliente), 0
UNION ALL SELECT 'dim_vendedor', (SELECT count(*) FROM stg.sellers), (SELECT count(*) FROM dw.dim_vendedor), 0
UNION ALL SELECT 'dim_producto', (SELECT count(*) FROM stg.products), (SELECT count(*) FROM dw.dim_producto), 0
UNION ALL SELECT 'dim_estado_pedido', (SELECT count(DISTINCT order_status) FROM stg.orders), (SELECT count(*) FROM dw.dim_estado_pedido), 0
UNION ALL SELECT 'dim_pago (5 tipos + sin_pago)', 6, (SELECT count(*) FROM dw.dim_pago), 0
UNION ALL SELECT 'dim_tiempo (2016-2018)', 1096, (SELECT count(*) FROM dw.dim_tiempo), 0;

\echo '=== V2. Integridad referencial: hechos sin dimensión (esperado 0) ==='
SELECT 'ventas sin cliente'   AS prueba, count(*) AS huerfanos FROM dw.fact_ventas_item f LEFT JOIN dw.dim_cliente  d USING (sk_cliente)  WHERE
d.sk_cliente  IS NULL
UNION ALL SELECT 'ventas sin vendedor',  count(*) FROM dw.fact_ventas_item f LEFT JOIN dw.dim_vendedor d USING (sk_vendedor) WHERE
d.sk_vendedor IS NULL
UNION ALL SELECT 'ventas sin producto',  count(*) FROM dw.fact_ventas_item f LEFT JOIN dw.dim_producto d USING (sk_producto) WHERE
d.sk_producto IS NULL
UNION ALL SELECT 'ventas sin fecha compra', count(*) FROM dw.fact_ventas_item f LEFT JOIN dw.dim_tiempo d ON d.sk_fecha = f.sk_fecha_compra
WHERE d.sk_fecha IS NULL
UNION ALL SELECT 'ventas con fecha entrega inexistente', count(*) FROM dw.fact_ventas_item f LEFT JOIN dw.dim_tiempo d ON d.sk_fecha =
f.sk_fecha_entrega WHERE f.sk_fecha_entrega IS NOT NULL AND d.sk_fecha IS NULL
UNION ALL SELECT 'pagos sin cliente', count(*) FROM dw.fact_pagos f LEFT JOIN dw.dim_cliente d USING (sk_cliente) WHERE d.sk_cliente IS NULL
UNION ALL SELECT 'pedidos con ítems en stg y sin fila en hechos', count(DISTINCT i.order_id)
      FROM stg.order_items i LEFT JOIN dw.fact_ventas_item f ON f.order_id = i.order_id WHERE f.order_id IS NULL;

\echo '=== V3. Unicidad de la granularidad (esperado 0 duplicados) ==='
SELECT count(*) - count(DISTINCT (order_id, order_item_id)) AS duplicados_en_fact_ventas FROM dw.fact_ventas_item;
SELECT count(*) - count(DISTINCT order_id) AS items_primer_item_extra FROM dw.fact_ventas_item WHERE es_primer_item = 1;

\echo '=== V4. Valores nulos relevantes en la tabla de hechos ==='
SELECT count(*) AS filas,
    count(*) FILTER (WHERE sk_fecha_entrega IS NULL) AS sin_fecha_entrega,
    count(*) FILTER (WHERE dias_entrega IS NULL)    AS sin_dias_entrega,
    count(*) FILTER (WHERE review_score IS NULL)    AS sin_resena,
    count(*) FILTER (WHERE precio IS NULL OR valor_flete IS NULL) AS sin_precio_o_flete
```

```
FROM dw.fact_ventas_item;

SELECT count(*) FILTER (WHERE sk_fecha_entrega IS NULL AND e.order_status = 'delivered') AS delivered_sin_fecha,
    count(*) FILTER (WHERE sk_fecha_entrega IS NULL AND e.order_status IN ('canceled','unavailable')) AS sin_fecha_cancel_o_no_disp,
    count(*) FILTER (WHERE sk_fecha_entrega IS NULL AND e.order_status NOT IN ('delivered','canceled','unavailable')) AS sin_fecha_en_proceso
FROM dw.fact_ventas_item f JOIN dw.dim_estado_pedido e USING (sk_estado);

SELECT count(*) FILTER (WHERE categoria_pt = 'sin_categoria') AS productos_sin_categoria,
    count(*) FILTER (WHERE peso_g IS NULL)              AS productos_sin_peso,
    (SELECT count(DISTINCT p.categoria_pt) FROM dw.dim_producto p
     WHERE p.categoria_pt <> 'sin_categoria'
       AND NOT EXISTS (SELECT 1 FROM stg.category_translation t WHERE t.product_category_name = p.categoria_pt))
                                        AS categorias_completadas_a_mano
FROM dw.dim_producto;

\echo '=== V5. Totales principales: dw vs consulta directa sobre staging ==='
SELECT 'GMV (sin cancelados)' AS medida,
    (SELECT round(sum(i.price::numeric),2) FROM stg.order_items i JOIN stg.orders o USING (order_id) WHERE o.order_status <> 'canceled') AS origen,
    (SELECT sum(precio) FROM dw.fact_ventas_item f JOIN dw.dim_estado_pedido e USING (sk_estado) WHERE NOT e.es_cancelado) AS dw
UNION ALL SELECT 'Flete total (todos)',
    (SELECT sum(freight_value::numeric) FROM stg.order_items), (SELECT sum(valor_flete) FROM dw.fact_ventas_item)
UNION ALL SELECT 'Valor de pagos (todos)',
    (SELECT sum(payment_value::numeric) FROM stg.order_payments), (SELECT sum(valor_pago) FROM dw.fact_pagos)
UNION ALL SELECT 'Pedidos con ítems',
    (SELECT count(DISTINCT order_id) FROM stg.order_items), (SELECT sum(es_primer_item) FROM dw.fact_ventas_item);

\echo '=== V6. Resumen de medidas y KPI ==='
SELECT dw.fn_total_ventas()         AS ventas_totales,
    dw.fn_total_pedidos()        AS pedidos,
    dw.fn_ticket_promedio()      AS ticket_prom,
    dw.fn_tiempo_entrega()       AS dias_entrega_prom,
    dw.fn_pct_entregas_tardias()  AS pct_tardias,
    dw.fn_calificacion_promedio() AS calif_prom,
    dw.fn_pct_resenas_negativas() AS pct_resenas_neg,
    dw.fn_participacion_flete()   AS pct_flete,
    dw.fn_pct_top10_vendedores()  AS pct_top10_vend,
    dw.fn_tasa_recompra()        AS tasa_recompra;

\echo '=== V7. Bitácora de calidad (por regla y acción) ==='
SELECT tabla, accion, count(*) AS registros,
    min(regla) AS ejemplo_regla
FROM dw.log_calidad GROUP BY tabla, accion ORDER BY registros DESC;
```

## 12.6 Salida de 04_validaciones.sql

Salida real de la ejecución sobre la base cargada con los 9 CSV originales.

```
=== V1. Cantidad de registros: origen (stg) vs destino (dw) ===
           tabla           | origen | destino | diferencia
-------------------------------+--------+---------+------------
 fact_ventas_item            | 112650 |  112650 |          0
 fact_pagos                  | 103886 |  103886 |          0
 dim_cliente                 |  99441 |   99441 |          0
 dim_vendedor                |   3095 |    3095 |          0
 dim_producto                |  32951 |   32951 |          0
 dim_estado_pedido           |      8 |       8 |          0
 dim_pago (5 tipos + sin_pago) |      6 |       6 |          0
 dim_tiempo (2016-2018)      |   1096 |    1096 |          0
(8 rows)

=== V2. Integridad referencial: hechos sin dimensión (esperado 0) ===
                 prueba                  | huerfanos
-----------------------------------------------------+-----------
 ventas sin fecha compra                 |         0
 ventas con fecha entrega inexistente    |         0
 ventas sin vendedor                     |         0
 ventas sin producto                     |         0
 pagos sin cliente                       |         0
 ventas sin cliente                      |         0
 pedidos con ítems en stg y sin fila en hechos |         0
(7 rows)

=== V3. Unicidad de la granularidad (esperado 0 duplicados) ===
 duplicados_en_fact_ventas
---------------------------
                         0
```

```
(1 row)

 items_primer_item_extra
-------------------------
                       0
(1 row)

=== V4. Valores nulos relevantes en la tabla de hechos ===
 filas  | sin_fecha_entrega | sin_dias_entrega | sin_resena | sin_precio_o_flete
--------+-------------------+------------------+------------+--------------------
 112650 |           2454 |          2454 |       942 |                  0
(1 row)

 delivered_sin_fecha | sin_fecha_cancel_o_no_disp | sin_fecha_en_proceso
---------------------+----------------------------+----------------------
                   8 |                        542 |                 1904
(1 row)

 productos_sin_categoria | productos_sin_peso | categorias_completadas_a_mano
-------------------------+--------------------+-------------------------------
                     610 |                  6 |                             2
(1 row)

=== V5. Totales principales: dw vs consulta directa sobre staging ===
        medida         |   origen    |     dw
-----------------------+-------------+-------------
 GMV (sin cancelados)   | 13496408.43 | 13496408.43
 Flete total (todos)    |  2251909.54 |  2251909.54
 Valor de pagos (todos) | 16008872.12 | 16008872.12
 Pedidos con ítems      |       98666 |       98666
(4 rows)

=== V6. Resumen de medidas y KPI ===
 ventas_totales | pedidos | ticket_prom | dias_entrega_prom | pct_tardias | calif_prom | pct_resenas_neg | pct_flete | pct_top10_vend | tasa_recompra
----------------+---------+-------------+-------------------+-------------+------------+-----------------+-----------+----------------+---------------
    13496408.43 |   98205 |      137.43 |             12.56 |        8.11 |       4.12 |           13.87 |     16.61 |          13.20 |          3.04
(1 row)

=== V7. Bitácora de calidad (por regla y acción) ===
     tabla      |  accion   | registros |                  ejemplo_regla
----------------+-----------+-----------+-------------------------------------------------------------------
 orders         | rechazado |       775 | pedido sin ítems: no tiene granularidad en FACT_VENTAS_ITEM
 products       | corregido |       614 | peso = 0 -> NULL
 order_reviews  | corregido |       547 | pedido con más de una reseña: se conserva la más reciente
 sellers        | corregido |        31 | ciudad con formato inválido normalizada
 orders         | marcado   |        14 | estado delivered sin fecha de entrega al cliente
 order_payments | marcado   |         9 | payment_value = 0
(6 rows)
```

# 13. Vistas y/o funciones SQL

El archivo 05_vistas_funciones.sql crea la capa de consumo. **La aplicación no consulta las tablas directamente:** solo usa estos objetos del esquema dw.

| **Objeto** | **Qué devuelve** |
|---|---|
| vw_ventas_detalle | El hecho unido a las 6 dimensiones (una fila por ítem). Es la base de todas las funciones. |
| vw_kpi_mensual | Ventas, pedidos, ticket y % de flete por mes. |
| vw_logistica_estado | Pedidos, días de entrega y % de tardías por estado del cliente. |
| vw_satisfaccion_entrega | Calificación y % de reseñas negativas con y sin retraso. |
| vw_pagos_resumen | Pagos, valor, pago promedio y cuotas por medio de pago. |
| fn_total_ventas(), fn_total_pedidos(), fn_ticket_promedio() | KPI comerciales (K1 a K3). |
| fn_tiempo_entrega(), fn_pct_entregas_tardias(), fn_participacion_flete() | KPI de logística (K4, K5, K8). |
| fn_calificacion_promedio(), fn_pct_resenas_negativas() | KPI de satisfacción (K6, K7). |
| fn_pct_top10_vendedores(), fn_tasa_recompra() | KPI de concentración y fidelidad (K9, K10). |
| fn_ventas_mensuales(), fn_top_categorias(), fn_logistica_por_estado(), fn_satisfaccion_entrega(), fn_pagos_resumen() | Series y rankings para los gráficos. |
| fn_ranking_vendedores(), fn_detalle_pedidos() | Tablas de detalle del dashboard. |

Las funciones de KPI reciben los mismos cuatro parámetros opcionales (NULL = sin filtro): p_desde, p_hasta, p_categoria (en inglés) y p_uf. Algunas funciones de gráfico omiten el parámetro que no tendría sentido (por ejemplo, el ranking de categorías no recibe la categoría).

### Ejemplos ejecutados

```
-- KPI sin filtros
SELECT dw.fn_total_ventas();                    -- 13496408.43
-- Los tres filtros combinados
SELECT dw.fn_total_ventas('2018-01-01','2018-08-31','health_beauty','SP');   -- 302498.32
SELECT dw.fn_ticket_promedio('2018-01-01','2018-08-31','health_beauty','SP'); -- 123.47
-- Función de tabla
SELECT * FROM dw.fn_top_categorias(NULL, NULL, NULL, 5);
```

| categoria | ventas_totales | pedidos | participacion_pct |
|---|---|---|---|
| health_beauty | 1.255.695,13 | 8.761 | 9,30 |
| watches_gifts | 1.198.185,21 | 5.581 | 8,88 |
| bed_bath_table | 1.035.964,06 | 9.293 | 7,68 |
| sports_leisure | 979.740,92 | 7.634 | 7,26 |
| computers_accessories | 904.322,02 | 6.625 | 6,70 |

## 13.1 sql/05_vistas_funciones.sql

```
-- ============================================================================
-- 05_vistas_funciones.sql  |  Capa de consumo para el dashboard
-- Convención: los KPI de pedido se calculan con es_primer_item = 1
-- (1 fila por pedido) para no duplicar por ítems. Porcentajes en escala 0-100.
-- ============================================================================

-- ---------------------------------------------------- vista base desnormalizada
CREATE OR REPLACE VIEW dw.vw_ventas_detalle AS
SELECT f.order_id, f.order_item_id, f.es_primer_item,
    tc.fecha AS fecha_compra, te.fecha AS fecha_entrega, tes.fecha AS fecha_estimada,
    tc.anio, tc.trimestre, tc.mes, tc.anio_mes,
    c.customer_unique_id, c.ciudad AS cliente_ciudad, c.estado_uf AS cliente_uf, c.region AS cliente_region,
    v.seller_id, v.ciudad AS vendedor_ciudad, v.estado_uf AS vendedor_uf,
    p.product_id, p.categoria_pt, p.categoria_en,
    pg.tipo_pago, e.order_status, e.es_entregado, e.es_cancelado,
    f.precio, f.valor_flete, f.valor_total_item,
    f.dias_entrega, f.dias_retraso, f.entrega_tardia, f.review_score
FROM dw.fact_ventas_item f
JOIN dw.dim_tiempo tc        ON tc.sk_fecha  = f.sk_fecha_compra
LEFT JOIN dw.dim_tiempo te   ON te.sk_fecha  = f.sk_fecha_entrega
JOIN dw.dim_tiempo tes       ON tes.sk_fecha = f.sk_fecha_estimada
JOIN dw.dim_cliente c        ON c.sk_cliente = f.sk_cliente
JOIN dw.dim_vendedor v       ON v.sk_vendedor = f.sk_vendedor
JOIN dw.dim_producto p       ON p.sk_producto = f.sk_producto
JOIN dw.dim_pago pg          ON pg.sk_pago   = f.sk_pago
JOIN dw.dim_estado_pedido e  ON e.sk_estado  = f.sk_estado;

-- ---------------------------------------------------- vistas agregadas
CREATE OR REPLACE VIEW dw.vw_kpi_mensual AS
SELECT anio_mes,
    round(sum(precio),2)                          AS ventas_totales,
    sum(es_primer_item)                           AS pedidos,
    round(sum(precio) / nullif(sum(es_primer_item),0), 2)      AS ticket_promedio,
    round(100 * sum(valor_flete) / nullif(sum(precio),0), 2)   AS pct_flete
FROM dw.vw_ventas_detalle WHERE NOT es_cancelado
GROUP BY anio_mes ORDER BY anio_mes;

CREATE OR REPLACE VIEW dw.vw_logistica_estado AS
SELECT cliente_uf, cliente_region,
    count(*)                                  AS pedidos_entregados,
    round(avg(dias_entrega),2)                AS dias_entrega_prom,
    round(100.0 * sum(entrega_tardia) / count(*), 2) AS pct_tardias
FROM dw.vw_ventas_detalle
WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL
```

```
GROUP BY cliente_uf, cliente_region ORDER BY pct_tardias DESC;

CREATE OR REPLACE VIEW dw.vw_satisfaccion_entrega AS
SELECT CASE WHEN entrega_tardia = 1 THEN 'Con retraso' ELSE 'A tiempo' END AS entrega,
    count(*)                    AS pedidos,
    round(avg(review_score),2)     AS calificacion_prom,
    round(100.0 * count(*) FILTER (WHERE review_score <= 2) / count(*), 2) AS pct_resenas_negativas
FROM dw.vw_ventas_detalle
WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL AND review_score IS NOT NULL
GROUP BY 1;

CREATE OR REPLACE VIEW dw.vw_pagos_resumen AS
SELECT d.tipo_pago, d.descripcion,
    count(*)                    AS pagos,
    count(DISTINCT p.order_id)       AS pedidos,
    round(sum(p.valor_pago),2)       AS valor_total,
    round(avg(p.valor_pago),2)       AS pago_promedio,
    round(avg(p.cuotas),2)           AS cuotas_promedio,
    max(p.cuotas)                    AS cuotas_max
FROM dw.fact_pagos p JOIN dw.dim_pago d USING (sk_pago)
JOIN dw.dim_estado_pedido e USING (sk_estado)
WHERE NOT e.es_cancelado
GROUP BY d.tipo_pago, d.descripcion ORDER BY valor_total DESC;

-- ---------------------------------------------------- funciones KPI (escalares)
-- Parámetros opcionales (NULL = sin filtro): fecha desde/hasta, categoría (inglés), UF del cliente.
CREATE OR REPLACE FUNCTION dw.fn_total_ventas(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(coalesce(sum(precio),0),2) FROM dw.vw_ventas_detalle
 WHERE NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_total_pedidos(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS bigint
LANGUAGE sql STABLE AS $$
 SELECT coalesce(sum(es_primer_item),0)::bigint FROM dw.vw_ventas_detalle
 WHERE NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_ticket_promedio(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(dw.fn_total_ventas(p_desde,p_hasta,p_categoria,p_uf)
      / nullif(dw.fn_total_pedidos(p_desde,p_hasta,p_categoria,p_uf),0), 2)
$$;

CREATE OR REPLACE FUNCTION dw.fn_tiempo_entrega(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(avg(dias_entrega),2) FROM dw.vw_ventas_detalle
 WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_pct_entregas_tardias(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(100.0 * sum(entrega_tardia) / nullif(count(*),0), 2) FROM dw.vw_ventas_detalle
 WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_calificacion_promedio(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(avg(review_score),2) FROM dw.vw_ventas_detalle
 WHERE es_primer_item = 1 AND review_score IS NOT NULL AND NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;
```

```
CREATE OR REPLACE FUNCTION dw.fn_pct_resenas_negativas(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(100.0 * count(*) FILTER (WHERE review_score <= 2) / nullif(count(*),0), 2)
 FROM dw.vw_ventas_detalle
 WHERE es_primer_item = 1 AND review_score IS NOT NULL AND NOT es_cancelado
  AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
  AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_participacion_flete(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 SELECT round(100 * sum(valor_flete) / nullif(sum(precio),0), 2) FROM dw.vw_ventas_detalle
 WHERE NOT es_cancelado
  AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
  AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
$$;

CREATE OR REPLACE FUNCTION dw.fn_pct_top10_vendedores(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 WITH v AS (
  SELECT seller_id, sum(precio) AS s FROM dw.vw_ventas_detalle
  WHERE NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
  GROUP BY seller_id)
 SELECT round(100 * (SELECT sum(s) FROM (SELECT s FROM v ORDER BY s DESC LIMIT 10) t)
        / nullif((SELECT sum(s) FROM v),0), 2)
$$;

CREATE OR REPLACE FUNCTION dw.fn_tasa_recompra(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL) RETURNS numeric
LANGUAGE sql STABLE AS $$
 WITH c AS (
  SELECT customer_unique_id, count(DISTINCT order_id) AS n FROM dw.vw_ventas_detalle
  WHERE NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
  GROUP BY customer_unique_id)
 SELECT round(100.0 * count(*) FILTER (WHERE n > 1) / nullif(count(*),0), 2) FROM c
$$;

-- ------------------------------------------------- funciones de tabla (gráficos y detalle)
CREATE OR REPLACE FUNCTION dw.fn_ventas_mensuales(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL)
RETURNS TABLE (anio_mes char(7), ventas_totales numeric, pedidos bigint, ticket_promedio numeric)
LANGUAGE sql STABLE AS $$
 SELECT v.anio_mes, round(sum(precio),2), sum(es_primer_item)::bigint,
      round(sum(precio)/nullif(sum(es_primer_item),0),2)
 FROM dw.vw_ventas_detalle v
 WHERE NOT es_cancelado
  AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
  AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
 GROUP BY v.anio_mes ORDER BY v.anio_mes
$$;

CREATE OR REPLACE FUNCTION dw.fn_top_categorias(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_uf text DEFAULT NULL, p_top int DEFAULT 10)
RETURNS TABLE (categoria text, ventas_totales numeric, pedidos bigint, participacion_pct numeric)
LANGUAGE sql STABLE AS $$
 WITH x AS (
  SELECT categoria_en AS cat, sum(precio) AS s, sum(es_primer_item) AS n FROM dw.vw_ventas_detalle
  WHERE NOT es_cancelado
   AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
   AND (p_uf IS NULL OR cliente_uf = p_uf)
  GROUP BY categoria_en)
 SELECT cat, round(s,2), n::bigint, round(100 * s / sum(s) OVER (), 2)
 FROM x ORDER BY s DESC LIMIT p_top
$$;

CREATE OR REPLACE FUNCTION dw.fn_logistica_por_estado(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
    p_categoria text DEFAULT NULL)
RETURNS TABLE (uf text, pedidos bigint, dias_entrega_prom numeric, pct_tardias numeric)
LANGUAGE sql STABLE AS $$
 SELECT cliente_uf, count(*), round(avg(dias_entrega),2), round(100.0*sum(entrega_tardia)/count(*),2)
 FROM dw.vw_ventas_detalle
```

```
  WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL
    AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
    AND (p_categoria IS NULL OR categoria_en = p_categoria)
  GROUP BY cliente_uf ORDER BY 4 DESC
$$;

CREATE OR REPLACE FUNCTION dw.fn_ranking_vendedores(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
      p_top int DEFAULT 10)
RETURNS TABLE (seller_id text, uf text, ventas_totales numeric, pedidos bigint, pct_tardias numeric, calificacion_prom numeric)
LANGUAGE sql STABLE AS $$
  SELECT v.seller_id, v.vendedor_uf, round(sum(precio),2), count(DISTINCT order_id),
       round(100.0 * sum(entrega_tardia) FILTER (WHERE es_entregado) / nullif(count(entrega_tardia) FILTER (WHERE es_entregado),0), 2),
       round(avg(review_score),2)
  FROM dw.vw_ventas_detalle v
  WHERE NOT es_cancelado
    AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
  GROUP BY v.seller_id, v.vendedor_uf ORDER BY sum(precio) DESC LIMIT p_top
$$;

CREATE OR REPLACE FUNCTION dw.fn_detalle_pedidos(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
      p_categoria text DEFAULT NULL, p_uf text DEFAULT NULL, p_limite int DEFAULT 100)
RETURNS TABLE (order_id text, fecha_compra date, cliente_uf text, categoria text, estado text,
            precio numeric, flete numeric, dias_entrega numeric, entrega_tardia smallint, review_score smallint)
LANGUAGE sql STABLE AS $$
  SELECT v.order_id, v.fecha_compra, v.cliente_uf, v.categoria_en, v.order_status,
       v.precio, v.valor_flete, v.dias_entrega, v.entrega_tardia, v.review_score
  FROM dw.vw_ventas_detalle v
  WHERE (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
    AND (p_categoria IS NULL OR categoria_en = p_categoria) AND (p_uf IS NULL OR cliente_uf = p_uf)
  ORDER BY v.fecha_compra DESC, v.order_id, v.order_item_id LIMIT p_limite
$$;

-- Satisfacción vs. puntualidad con filtros (alimenta los gráficos C10 y C11)
CREATE OR REPLACE FUNCTION dw.fn_satisfaccion_entrega(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL,
      p_categoria text DEFAULT NULL)
RETURNS TABLE (entrega text, pedidos bigint, calificacion_prom numeric, pct_resenas_negativas numeric)
LANGUAGE sql STABLE AS $$
  SELECT CASE WHEN entrega_tardia = 1 THEN 'Con retraso' ELSE 'A tiempo' END,
       count(*), round(avg(review_score),2),
       round(100.0 * count(*) FILTER (WHERE review_score <= 2) / count(*), 2)
  FROM dw.vw_ventas_detalle
  WHERE es_primer_item = 1 AND es_entregado AND fecha_entrega IS NOT NULL AND review_score IS NOT NULL
    AND (p_desde IS NULL OR fecha_compra >= p_desde) AND (p_hasta IS NULL OR fecha_compra <= p_hasta)
    AND (p_categoria IS NULL OR categoria_en = p_categoria)
  GROUP BY 1 ORDER BY 1
$$;

-- Medios de pago y cuotas con filtro de fecha (alimenta los gráficos C19 y C20)
CREATE OR REPLACE FUNCTION dw.fn_pagos_resumen(p_desde date DEFAULT NULL, p_hasta date DEFAULT NULL)
RETURNS TABLE (tipo_pago text, descripcion text, pagos bigint, pedidos bigint, valor_total numeric,
            pago_promedio numeric, cuotas_promedio numeric, cuotas_max smallint)
LANGUAGE sql STABLE AS $$
  SELECT d.tipo_pago, d.descripcion, count(*), count(DISTINCT p.order_id),
       round(sum(p.valor_pago),2), round(avg(p.valor_pago),2), round(avg(p.cuotas),2), max(p.cuotas)
  FROM dw.fact_pagos p
  JOIN dw.dim_pago d ON d.sk_pago = p.sk_pago
  JOIN dw.dim_estado_pedido e ON e.sk_estado = p.sk_estado
  JOIN dw.dim_tiempo t ON t.sk_fecha = p.sk_fecha_compra
  WHERE NOT e.es_cancelado
    AND (p_desde IS NULL OR t.fecha >= p_desde) AND (p_hasta IS NULL OR t.fecha <= p_hasta)
  GROUP BY d.tipo_pago, d.descripcion ORDER BY 5 DESC
$$;
```

# 14. Mockup

El mockup del Entregable 2 representa el dashboard en tres pestañas, una por área usuaria, con una barra de filtros común. Sus valores no son inventados: se obtuvieron llamando a las funciones del Data Mart con el rango 2017-01-01 a 2018-08-31 (el período de mayor volumen). El archivo interactivo es docs/Mockup_Dashboard_Olist.html.

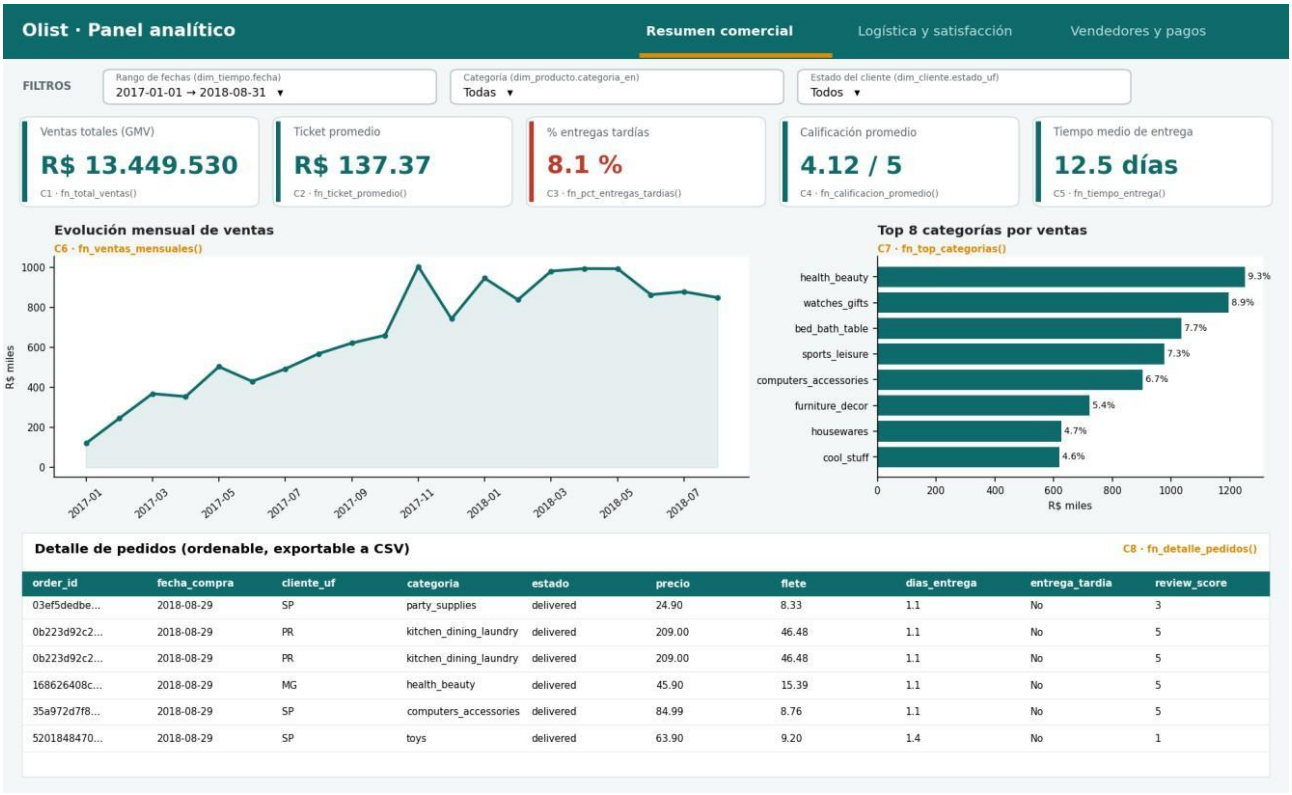



*Figura 6. Mockup, pestaña 1: Resumen comercial.*

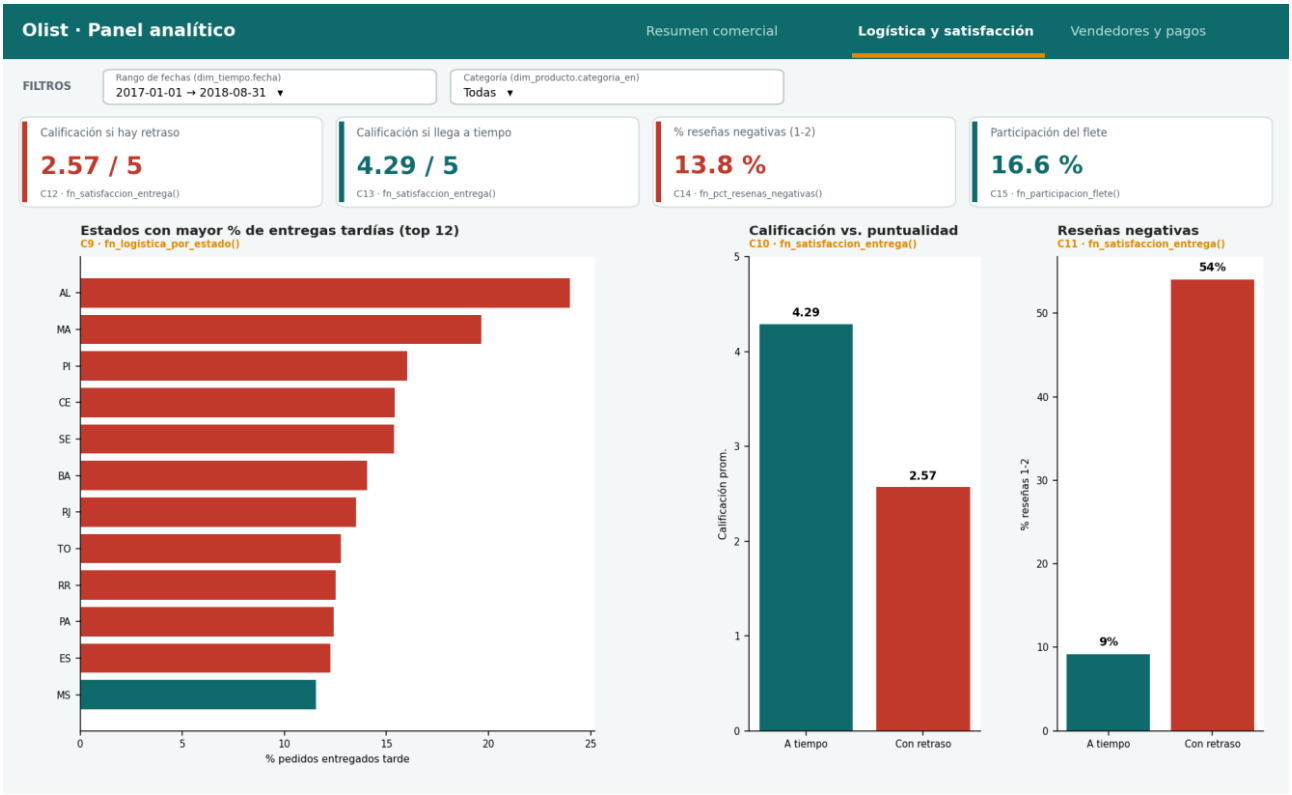



*Figura 7. Mockup, pestaña 2: Logística y satisfacción.*

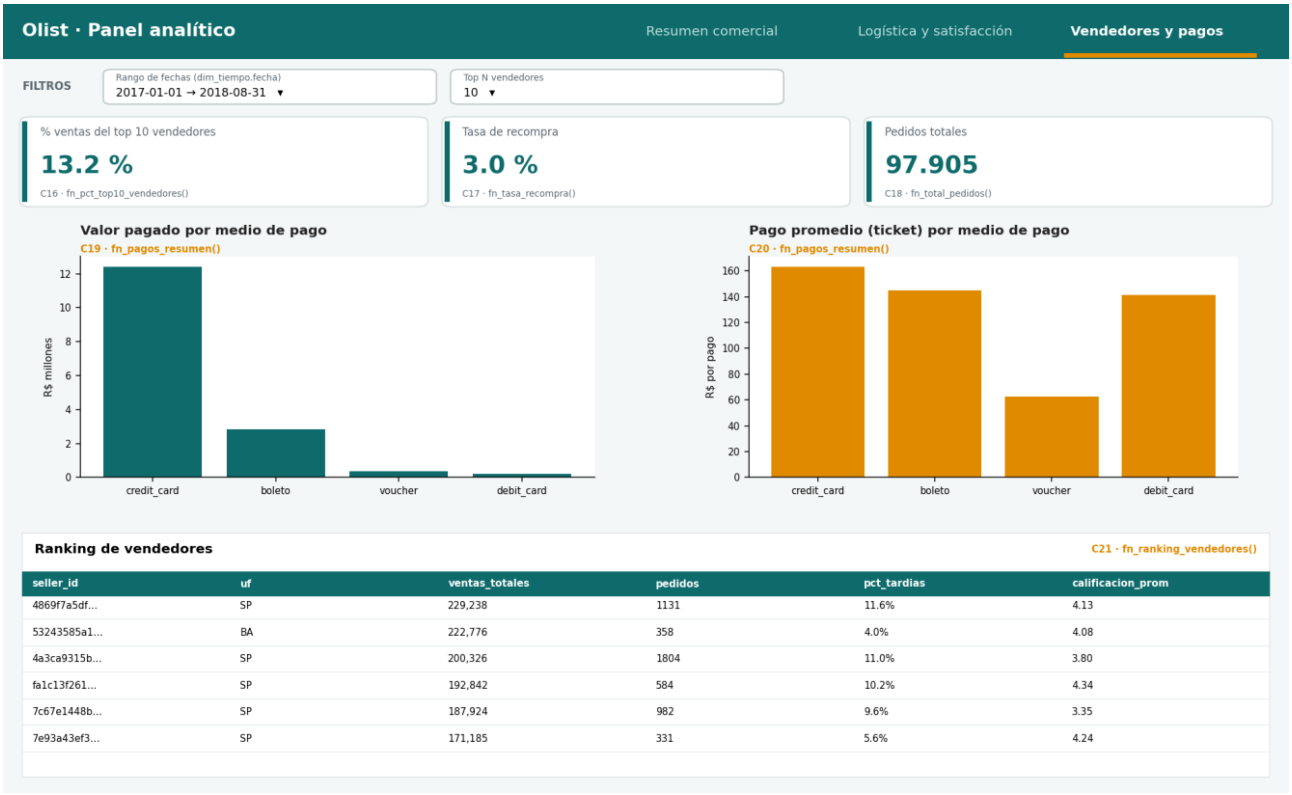



*Figura 8. Mockup, pestaña 3: Vendedores y pagos.*

### 14.1 Del mockup a la aplicación implementada

La aplicación final conserva la estructura del mockup (tres áreas, tres filtros, KPI, gráficos y tabla). Estas son las diferencias reales entre ambos:

| Elemento | Mockup (Entregable 2) | Aplicación final |
|---|---|---|
| Filtros | Barra superior con tres controles | Panel lateral con los mismos tres filtros (fecha, categoría, estado UF), enviados como parámetros a las funciones SQL |
| Navegación | Tres pestañas | Selector horizontal de tres vistas |
| KPI de «Resumen comercial» | 5 tarjetas: GMV, ticket, % tardías, calificación y tiempo de entrega | 4 tarjetas: GMV, pedidos, ticket y participación del flete. Los KPI de entrega y calificación se ubican en «Logística y satisfacción», que es el área usuaria que los usa |
| Satisfacción vs. puntualidad | Dos gráficos y dos tarjetas (calificación con y sin retraso) | Un gráfico de calificación según puntualidad y una tabla con pedidos, calificación y % de reseñas negativas |
| Top de categorías | Fijo en 8 | Ajustable con un control deslizante (3 a 15) |
| Detalle de pedidos | Tabla ordenable y exportable | Tabla con número de filas seleccionable (25 a 1.000) y botón «Descargar CSV» |
| Transparencia | Códigos C1 a C21 impresos junto a cada componente | Desplegable «Ver consulta SQL utilizada» en cada vista; matriz de trazabilidad de la sección 15 |
| Meses incompletos | No señalado | Nota bajo el gráfico: la caída final (septiembre de 2018) refleja datos parciales del dataset, no una caída real de ventas |
| Filtros que no aplican | No señalado | Cada sección indica qué filtro no usa su consulta (por ejemplo, el ranking de categorías no usa el de categoría) |

# 15. Matriz de trazabilidad

La solución mantiene la relación entre las necesidades de negocio (preguntas y KPI), los elementos visuales de la aplicación y los datos. Los filtros se definen una sola vez:

| Filtro | Control | Columna del Data Mart | Parámetro SQL | Lista de valores (consulta en db.py) |
|---|---|---|---|---|
| F1 | Rango de fechas | vw_ventas_detalle.fecha_compra (de dim_tiempo.fecha) | p_desde, p_hasta | SELECT min(fecha_compra), max(fecha_compra) FROM dw.vw_ventas_detalle |

| | | | | |
|---|---|---|---|---|
| F2 | Categoría de producto | dim_producto.categoria_en | p_categoria | SELECT DISTINCT categoria_en FROM dw.dim_producto ORDER BY 1 |
| F3 | Estado del cliente (UF) | dim_cliente.estado_uf | p_uf | SELECT DISTINCT estado_uf FROM dw.dim_cliente ORDER BY 1 |

| Cód. | Componente | Pregunta / KPI | Filtros | Datos utilizados | Vista / función SQL |
|---|---|---|---|---|---|
| **Vista 1 · Resumen comercial** | | | | | |
| A 1 | Tarjeta · Ventas totales (GMV) | P1 · K1 | F1, F2, F3 | fact_ventas_item.precio + dim_tiempo, dim_producto, dim_cliente, dim_estado_pedido | dw.fn_total_ventas() |
| A 2 | Tarjeta · Pedidos | P1 · K2 | F1, F2, F3 | fact_ventas_item.es_primer_item + mismas dimensiones | dw.fn_total_pedidos() |
| A 3 | Tarjeta · Ticket promedio | P1, P4 · K3 | F1, F2, F3 | precio, es_primer_item | dw.fn_ticket_promedio() |
| A 4 | Tarjeta · Participación del flete | P1, P2 · K8 | F1, F2, F3 | valor_flete, precio | dw.fn_participacion_flete() |
| A 5 | Línea · Evolución mensual de ventas | P1 | F1, F2, F3 | precio, es_primer_item + dim_tiempo.anio_mes | dw.fn_ventas_mensuales() (sin filtros: dw.vw_kpi_mensual) |
| A 6 | Barras · Top N categorías | P1 | F1, F3 (+ N) | precio + dim_producto.categoria_en | dw.fn_top_categorias() |
| A 7 | Tabla · Detalle de pedidos | P1, P2, P3 | F1, F2, F3 (+ filas) | vw_ventas_detalle (hecho + 6 dimensiones) | dw.fn_detalle_pedidos() |
| **Vista 2 · Logística y satisfacción** | | | | | |
| A 8 | Tarjeta · Tiempo medio de entrega | P2 · K4 | F1, F2, F3 | dias_entrega + dim_estado_pedido.es_entregado | dw.fn_tiempo_entrega() |
| A 9 | Tarjeta · % entregas tardías | P2, P3 · K5 | F1, F2, F3 | entrega_tardia, es_primer_item | dw.fn_pct_entregas_tardias() |
| A 10 | Tarjeta · Calificación promedio | P3 · K6 | F1, F2, F3 | review_score, es_primer_item | dw.fn_calificacion_promedio() |
| A 11 | Tarjeta · % reseñas negativas (1-2) | P3 · K7 | F1, F2, F3 | review_score ≤ 2 | dw.fn_pct_resenas_negativas() |
| A 12 | Barras · % tardías por estado | P2 | F1, F2 (+ N estados) | entrega_tardia, dias_entrega + dim_cliente.estado_uf | dw.fn_logistica_por_estado() (sin filtros: dw.vw_logistica_estado) |
| A 13 | Barras · Calificación según puntualidad | P3 | F1, F2 | entrega_tardia, review_score | dw.fn_satisfaccion_entrega() (sin filtros: dw.vw_satisfaccion_entrega) |
| A 14 | Tabla · Pedidos, calificación y % negativas por puntualidad | P3 | F1, F2 | entrega_tardia, review_score | dw.fn_satisfaccion_entrega() |
| **Vista 3 · Vendedores y pagos** | | | | | |
| A 15 | Tarjeta · % ventas del top 10 de vendedores | P5 · K9 | F1, F2, F3 | precio + dim_vendedor.seller_id | dw.fn_pct_top10_vendedores() |
| A 16 | Tarjeta · Tasa de recompra | P5 · K10 | F1, F2, F3 | dim_cliente.customer_unique_id | dw.fn_tasa_recompra() |
| A 17 | Tarjeta · Pedidos | P1, P5 · K2 | F1, F2, F3 | es_primer_item | dw.fn_total_pedidos() |
| A 18 | Barras · Valor pagado por medio de pago | P4 | F1 | fact_pagos.valor_pago + dim_pago, dim_tiempo, dim_estado_pedido | dw.fn_pagos_resumen() (sin filtros: dw.vw_pagos_resumen) |
| A 19 | Barras · Pago promedio por medio de pago (cuotas en el tooltip) | P4 | F1 | fact_pagos.valor_pago, cuotas | dw.fn_pagos_resumen() |
| A 20 | Tabla · Ranking de vendedores | P5 | F1 (+ N) | precio, entrega_tardia, review_score + dim_vendedor | dw.fn_ranking_vendedores() |

Los filtros que no aplican a una consulta (por ejemplo, F2 y F3 en los medios de pago) se omiten a propósito: la función SQL no los recibe y la aplicación lo avisa bajo cada gráfico.

# 16. Código fuente Python

Código fuente Python del proyecto. La aplicación (app/app.py y app/db.py) se describe en la sección 17; python/perfilado.py es el perfilado inicial de los CSV usado en el Entregable 1.

## 16.1 app/db.py

```
"""Acceso a PostgreSQL. TODOS los datos del dashboard salen del Data Mart (esquema dw):
la aplicación nunca lee los CSV originales."""
import os

import pandas as pd
import streamlit as st
from sqlalchemy import create_engine, text
```

```
@st.cache_resource
def get_engine():
    """Una sola conexión (pool) reutilizada por toda la aplicación.
    Los valores por defecto coinciden con docker-compose.yml; pueden cambiarse con variables de entorno."""
    url = "postgresql+psycopg2://{u}:{p}@{h}:{port}/{d}".format(
        u=os.getenv("PGUSER", "bi_user"),
        p=os.getenv("PGPASSWORD", "bi_password"),
        h=os.getenv("PGHOST", "localhost"),
        port=os.getenv("PGPORT", "5432"),
        d=os.getenv("PGDATABASE", "bi_database"),
    )
    return create_engine(url, pool_pre_ping=True)


@st.cache_data(ttl=300, show_spinner=False)
def run(sql: str, params: dict | None = None) -> pd.DataFrame:
    """Ejecuta una consulta con parámetros (evita inyección SQL) y devuelve un DataFrame."""
    with get_engine().connect() as conn:
        return pd.read_sql(text(sql), conn, params=params or {})


# Los cuatro filtros viajan como parámetros :d (desde), :h (hasta), :c (categoría), :u (estado).
# NULL = sin filtro, igual que en las funciones SQL.
ARGS = "CAST(:d AS date), CAST(:h AS date), CAST(:c AS text), CAST(:u AS text)"
ARGS_FECHA_CAT = "CAST(:d AS date), CAST(:h AS date), CAST(:c AS text)"
ARGS_FECHA = "CAST(:d AS date), CAST(:h AS date)"

SQL = {
    "opciones_fechas": "SELECT min(fecha_compra) AS desde, max(fecha_compra) AS hasta FROM dw.vw_ventas_detalle",
    "opciones_categorias": "SELECT DISTINCT categoria_en AS v FROM dw.dim_producto ORDER BY 1",
    "opciones_uf": "SELECT DISTINCT estado_uf AS v FROM dw.dim_cliente ORDER BY 1",
    "kpi_comercial": f"""SELECT dw.fn_total_ventas({ARGS}) AS ventas, dw.fn_total_pedidos({ARGS}) AS pedidos,
        dw.fn_ticket_promedio({ARGS}) AS ticket, dw.fn_participacion_flete({ARGS}) AS flete""",
    "ventas_mensuales": f"SELECT * FROM dw.fn_ventas_mensuales({ARGS})",
    "top_categorias": f"SELECT * FROM dw.fn_top_categorias(CAST(:d AS date), CAST(:h AS date), CAST(:u AS text), :n)",
    "detalle": f"SELECT * FROM dw.fn_detalle_pedidos({ARGS}, :n)",
    "kpi_logistica": f"""SELECT dw.fn_tiempo_entrega({ARGS}) AS dias, dw.fn_pct_entregas_tardias({ARGS}) AS tardias,
        dw.fn_calificacion_promedio({ARGS}) AS calificacion, dw.fn_pct_resenas_negativas({ARGS}) AS negativas""",
    "logistica_estado": f"SELECT * FROM dw.fn_logistica_por_estado({ARGS_FECHA_CAT})",
    "satisfaccion": f"SELECT * FROM dw.fn_satisfaccion_entrega({ARGS_FECHA_CAT})",
    "kpi_vendedores": f"""SELECT dw.fn_pct_top10_vendedores({ARGS}) AS top10, dw.fn_tasa_recompra({ARGS}) AS recompra,
        dw.fn_total_pedidos({ARGS}) AS pedidos""",
    "pagos": f"SELECT * FROM dw.fn_pagos_resumen({ARGS_FECHA})",
    "ranking": f"SELECT * FROM dw.fn_ranking_vendedores({ARGS_FECHA}, :n)",
}


def consulta(nombre: str, filtros: dict, **extra) -> pd.DataFrame:
    return run(SQL[nombre], {**filtros, **extra})
```

## 16.2 app/app.py

```
"""Dashboard de Business Intelligence · Olist Marketplace
Ejecutar:  streamlit run app/app.py
"""
import pandas as pd
import plotly.express as px
import streamlit as st

import db

st.set_page_config(page_title="Olist · Panel analítico", page_icon=" 📊 ", layout="wide")
TEAL, ORANGE, RED = "#0F6B6B", "#C77700", "#B5352A"


def num(v, dec=0, pre="", suf=""):
    """Formato con punto de miles y coma decimal (1.234,56). None -> guion."""
    if v is None or pd.isna(v):
        return "—"
    s = f"{v:,.{dec}f}".replace(",", "X").replace(".", ",").replace("X", ".")
    return f"{pre}{s}{suf}"


# ---------------------------------------------------------------------- conexión
try:
```

```
    fechas = db.run(db.SQL["opciones_fechas"]).iloc[0]
    categorias = db.run(db.SQL["opciones_categorias"])["v"].tolist()
    ufs = db.run(db.SQL["opciones_uf"])["v"].tolist()
except Exception as e:  # base apagada, credenciales erróneas, datos sin cargar
    st.error("No se pudo consultar PostgreSQL. Verifica que el contenedor esté activo "
             "(`docker compose up -d`) y que los scripts SQL 01 a 05 se hayan ejecutado.")
    st.code(str(e)[:400])
    st.stop()

# ---------------------------------------------------------------------- filtros
st.sidebar.title("Filtros")
rango = st.sidebar.date_input("Fecha de compra", value=(fechas["desde"], fechas["hasta"]),
                              min_value=fechas["desde"], max_value=fechas["hasta"])
cat = st.sidebar.selectbox("Categoría de producto", ["Todas"] + categorias)
uf = st.sidebar.selectbox("Estado del cliente (UF)", ["Todos"] + ufs)
if not (isinstance(rango, (tuple, list)) and len(rango) == 2):
    st.info("Selecciona la fecha de fin del rango en el panel lateral.")
    st.stop()
F = {"d": rango[0], "h": rango[1], "c": None if cat == "Todas" else cat, "u": None if uf == "Todos" else uf}
st.sidebar.caption("Los filtros se envían como parámetros a las funciones SQL del esquema `dw`. "
                   "Dónde un filtro no aplica a una consulta, se indica en la sección.")

st.title("📊 Olist · Panel analítico")
vista = st.radio("Vista", ["Resumen comercial", "Logística y satisfacción", "Vendedores y pagos"],
                 horizontal=True, label_visibility="collapsed")


def sql_expander(*nombres):
    with st.expander("Ver consulta SQL utilizada"):
        for n in nombres:
            st.code(db.SQL[n], language="sql")
        st.caption(f"Parámetros actuales: {F}")


def sin_datos(df):
    if df.empty:
        st.info("No hay datos para los filtros seleccionados.")
        return True
    return False


# ---------------------------------------------------------------- 1. resumen comercial
if vista == "Resumen comercial":
    k = db.consulta("kpi_comercial", F).iloc[0]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Ventas totales (GMV)", num(k.ventas, 0, "R$ "))
    c2.metric("Pedidos", num(k.pedidos))
    c3.metric("Ticket promedio", num(k.ticket, 2, "R$ "))
    c4.metric("Participación del flete", num(k.flete, 2, suf=" %"))

    izq, der = st.columns([3, 2])
    with izq:
        st.subheader("Evolución mensual de ventas")
        m = db.consulta("ventas_mensuales", F)
        if not sin_datos(m):
            fig = px.line(m, x="anio_mes", y="ventas_totales", markers=True,
                          labels={"anio_mes": "Mes", "ventas_totales": "Ventas (R$)"})
            fig.update_traces(line_color=TEAL)
            st.plotly_chart(fig, width="stretch")
            st.caption("Los meses extremos del dataset están incompletos (finales de 2016 y septiembre-octubre de 2018): la caída final refleja datos parciales, no una caída real de ventas.")
    with der:
        n = st.slider("Categorías a mostrar", 3, 15, 8)
        st.subheader(f"Top {n} categorías")
        t = db.consulta("top_categorias", F, n=n)
        if not sin_datos(t):
            fig = px.bar(t.iloc[::-1], x="ventas_totales", y="categoria", orientation="h",
                         text="participacion_pct", labels={"ventas_totales": "Ventas (R$)", "categoria": ""})
            fig.update_traces(marker_color=TEAL, texttemplate="%{text:.1f}%")
            st.plotly_chart(fig, width="stretch")
        st.caption("El ranking compara categorías entre sí, por eso usa el rango de fechas y el estado, pero no el filtro de categoría.")

    st.subheader("Detalle de pedidos")
    filas = st.select_slider("Filas a mostrar", [25, 50, 100, 500, 1000], value=100)
    d = db.consulta("detalle", F, n=filas)
    if not sin_datos(d):
        st.dataframe(d, hide_index=True, width="stretch")
        st.download_button("Descargar CSV", d.to_csv(index=False).encode("utf-8"), "detalle_pedidos.csv", "text/csv")
```

```
    sql_expander("kpi_comercial", "ventas_mensuales", "top_categorias", "detalle")

# ---------------------------------------------------- 2. logística y satisfacción
elif vista == "Logística y satisfacción":
    k = db.consulta("kpi_logistica", F).iloc[0]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Tiempo medio de entrega", num(k.dias, 1, suf=" días"))
    c2.metric("% entregas tardías", num(k.tardias, 2, suf=" %"))
    c3.metric("Calificación promedio", num(k.calificacion, 2, suf=" / 5"))
    c4.metric("% reseñas negativas (1-2)", num(k.negativas, 2, suf=" %"))

    izq, der = st.columns([3, 2])
    with izq:
        st.subheader("Entregas tardías por estado del cliente")
        e = db.consulta("logistica_estado", F)
        if not sin_datos(e):
            top = st.slider("Estados a mostrar", 5, 27, 12)
            e = e.head(top).iloc[::-1]
            fig = px.bar(e, x="pct_tardias", y="uf", orientation="h", hover_data=["pedidos", "dias_entrega_prom"],
                         labels={"pct_tardias": "% pedidos tardíos", "uf": "Estado"})
            fig.update_traces(marker_color=RED)
            st.plotly_chart(fig, width="stretch")
        st.caption("Esta consulta agrupa por estado, por eso no usa el filtro de estado (UF).")
    with der:
        st.subheader("Calificación según puntualidad")
        s = db.consulta("satisfaccion", F)
        if not sin_datos(s):
            fig = px.bar(s, x="entrega", y="calificacion_prom", text="calificacion_prom",
                         color="entrega", color_discrete_map={"A tiempo": TEAL, "Con retraso": RED},
                         labels={"entrega": "", "calificacion_prom": "Calificación promedio"})
            fig.update_layout(showlegend=False, yaxis_range=[0, 5])
            st.plotly_chart(fig, width="stretch")
            st.dataframe(s.rename(columns={"calificacion_prom": "calificación prom.",
                                           "pct_resenas_negativas": "% reseñas negativas"}),
                         hide_index=True, width="stretch")
    sql_expander("kpi_logistica", "logistica_estado", "satisfaccion")

# ---------------------------------------------------- 3. vendedores y pagos
else:
    k = db.consulta("kpi_vendedores", F).iloc[0]
    c1, c2, c3 = st.columns(3)
    c1.metric("% ventas del top 10 de vendedores", num(k.top10, 2, suf=" %"))
    c2.metric("Tasa de recompra", num(k.recompra, 2, suf=" %"))
    c3.metric("Pedidos", num(k.pedidos))

    st.subheader("Medios de pago")
    p = db.consulta("pagos", F)
    if not sin_datos(p):
        a, b = st.columns(2)
        fig = px.bar(p, x="tipo_pago", y="valor_total", labels={"tipo_pago": "", "valor_total": "Valor pagado (R$)"})
        fig.update_traces(marker_color=TEAL)
        a.plotly_chart(fig, width="stretch")
        fig = px.bar(p, x="tipo_pago", y="pago_promedio", hover_data=["cuotas_promedio", "cuotas_max"],
                     labels={"tipo_pago": "", "pago_promedio": "Pago promedio (R$)"})
        fig.update_traces(marker_color=ORANGE)
        b.plotly_chart(fig, width="stretch")
    st.caption("Medios de pago y ranking usan el rango de fechas; los filtros de categoría y estado no aplican a estas consultas.")

    st.subheader("Ranking de vendedores")
    n = st.slider("Vendedores a mostrar", 5, 50, 10)
    r = db.consulta("ranking", F, n=n)
    if not sin_datos(r):
        st.dataframe(r, hide_index=True, width="stretch")
    sql_expander("kpi_vendedores", "pagos", "ranking")
```

## 16.3 app/requirements.txt

```
streamlit>=1.40
pandas>=2.0
plotly>=5.20
SQLAlchemy>=2.0
psycopg2-binary>=2.9
```

### 16.4 python/perfilado.py (perfilado inicial de los CSV)

```
import pandas as pd, glob

for f in sorted(glob.glob("*.csv")):
  df = pd.read_csv(f)
  print(f, df.shape, "duplicadas:", df.duplicated().sum())
  print(df.isna().sum()[df.isna().sum() > 0])

o = pd.read_csv("olist_orders_dataset.csv", parse_dates=["order_purchase_timestamp"])
print(o.order_purchase_timestamp.min(), o.order_purchase_timestamp.max())
print(o.order_status.value_counts())

c = pd.read_csv("olist_customers_dataset.csv")
print(c.customer_unique_id.nunique())
```

# 17. Proyecto Streamlit

## 17.1 Arquitectura

PostgreSQL (esquema dw: vistas y funciones)
↓ SQLAlchemy + psycopg2 (consultas parametrizadas) app/db.py
pandas DataFrame
↓ app/app.py
Streamlit + Plotly → filtros, KPI, gráficos y tablas

- **Conexión (app/db.py).** get_engine() crea una sola conexión con pool (@st.cache_resource) con los datos de docker-compose.yml; pueden cambiarse con las variables de entorno PGHOST, PGPORT, PGDATABASE, PGUSER y PGPASSWORD.
- **Consultas (run()).** Ejecuta SQL con parámetros nombrados (:d, :h, :c, :u), lo que evita inyección SQL, y devuelve un DataFrame con caché de 5 minutos. El diccionario SQL concentra todas las consultas que usa la aplicación.
- **Solo PostgreSQL.** La aplicación no abre ningún CSV de origen: todos los datos salen de las vistas y funciones del esquema dw. (El único CSV que genera es la descarga opcional del detalle de pedidos.)
- **Filtros (app/app.py).** Tres controles en el panel lateral —rango de fechas de compra, categoría de producto y estado (UF) del cliente— se empaquetan en el diccionario F y viajan como parámetros a las funciones SQL; por eso cambian los resultados. Donde un filtro no aplica a una consulta, la aplicación lo indica en la sección.
- **Transparencia.** Cada vista tiene el desplegable «Ver consulta SQL utilizada», que muestra el SQL y los parámetros vigentes.

## 17.2 Ejecución

```
python -m venv .venv
.venv\Scripts\activate          # Windows  (Linux/Mac: source .venv/bin/activate)
pip install -r app/requirements.txt   # streamlit, pandas, plotly, SQLAlchemy, psycopg2-binary
streamlit run app/app.py          # http://localhost:8501
```

## 17.3 Contenido de las tres vistas

| Vista | KPI | Gráficos y detalle |
|---|---|---|
| Resumen comercial | Ventas totales, pedidos, ticket promedio, % de flete | Evolución mensual, top N de categorías, tabla de detalle descargable |
| Logística y satisfacción | Tiempo de entrega, % tardías, calificación, % reseñas negativas | Tardías por estado, calificación según puntualidad |
| Vendedores y pagos | % ventas top 10, tasa de recompra, pedidos | Valor y pago promedio por medio de pago, ranking de vendedores |

Capturas tomadas de la aplicación en ejecución, conectada a la base cargada (rango completo 2016-09-04 a 2018-09-03, sin filtros salvo indicación):

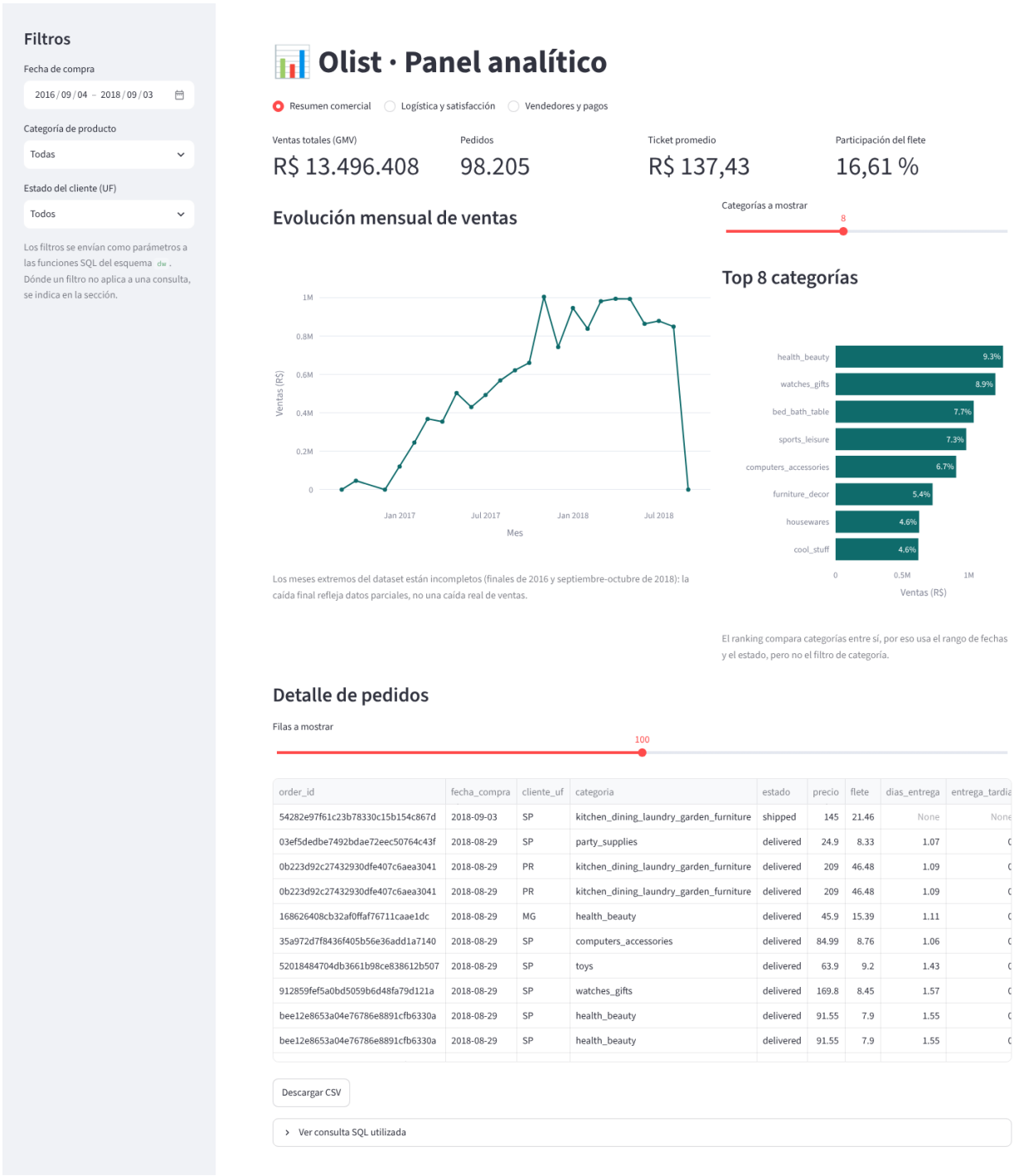



*Figura 2. Vista «Resumen comercial» sin filtros: GMV R$ 13.496.408, 98.205 pedidos, ticket R$ 137,43 y flete 16,61 %.*

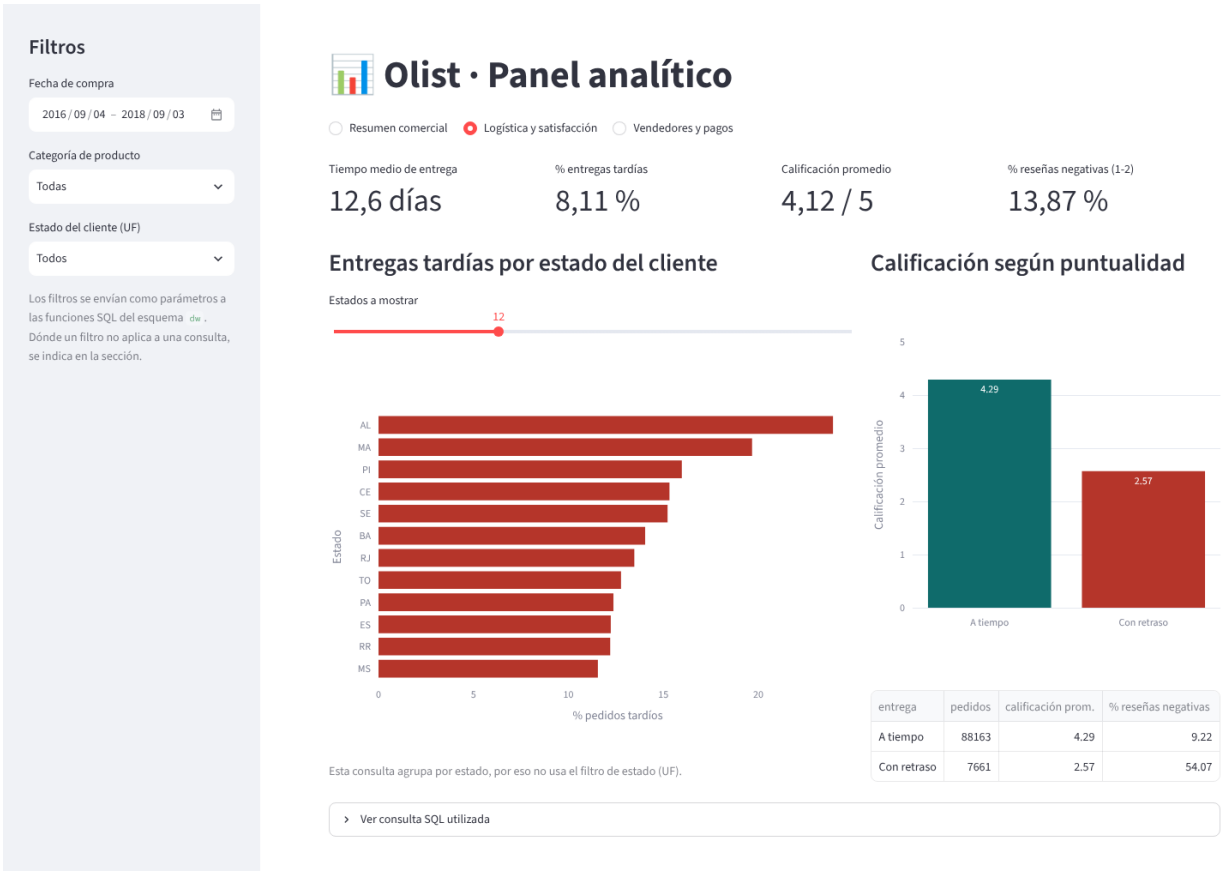



*Figura 3. Vista «Logística y satisfacción»: 12,6 días, 8,11 % de tardías, calificación 4,12 y 13,87 % de reseñas negativas.*

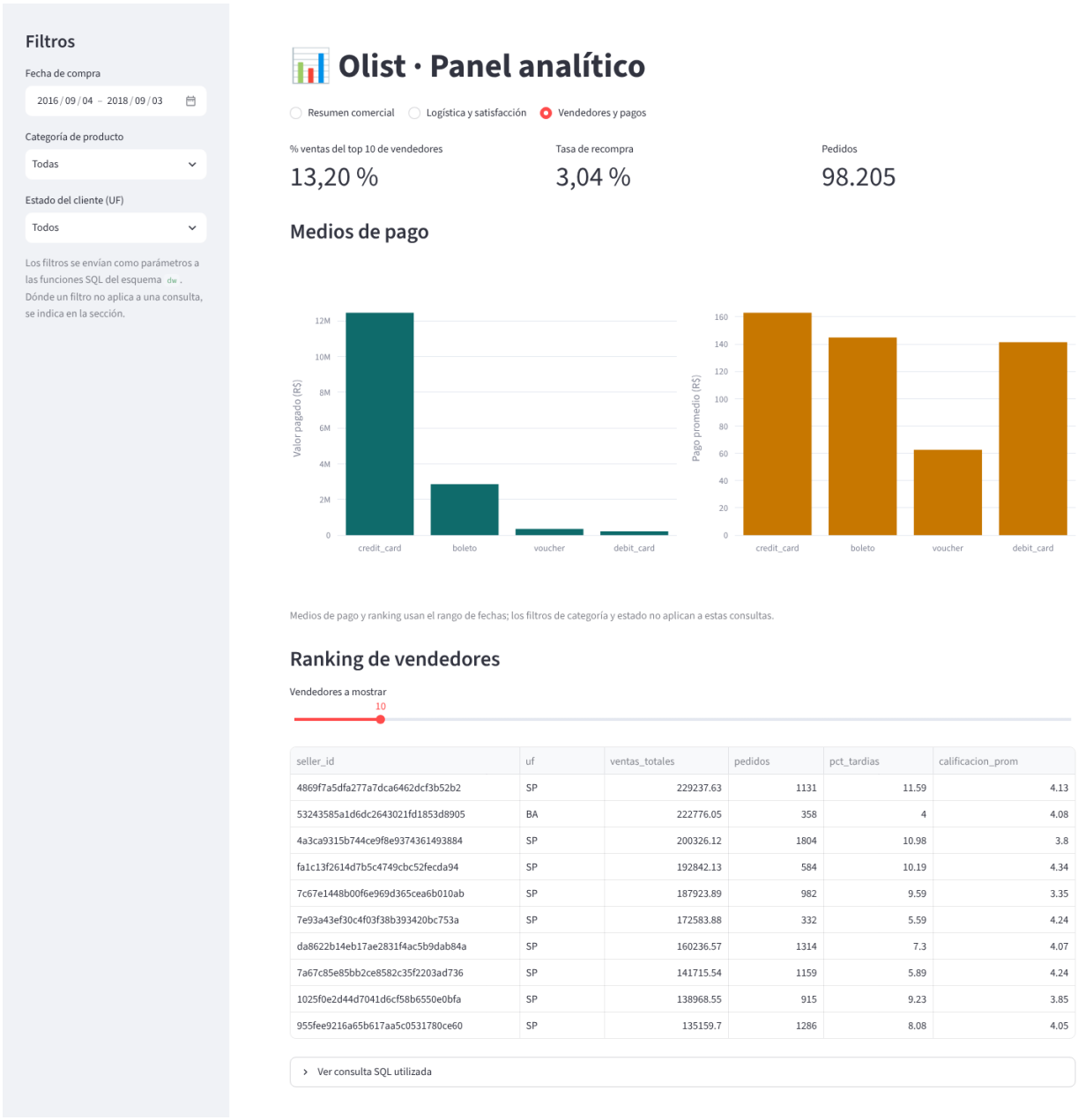
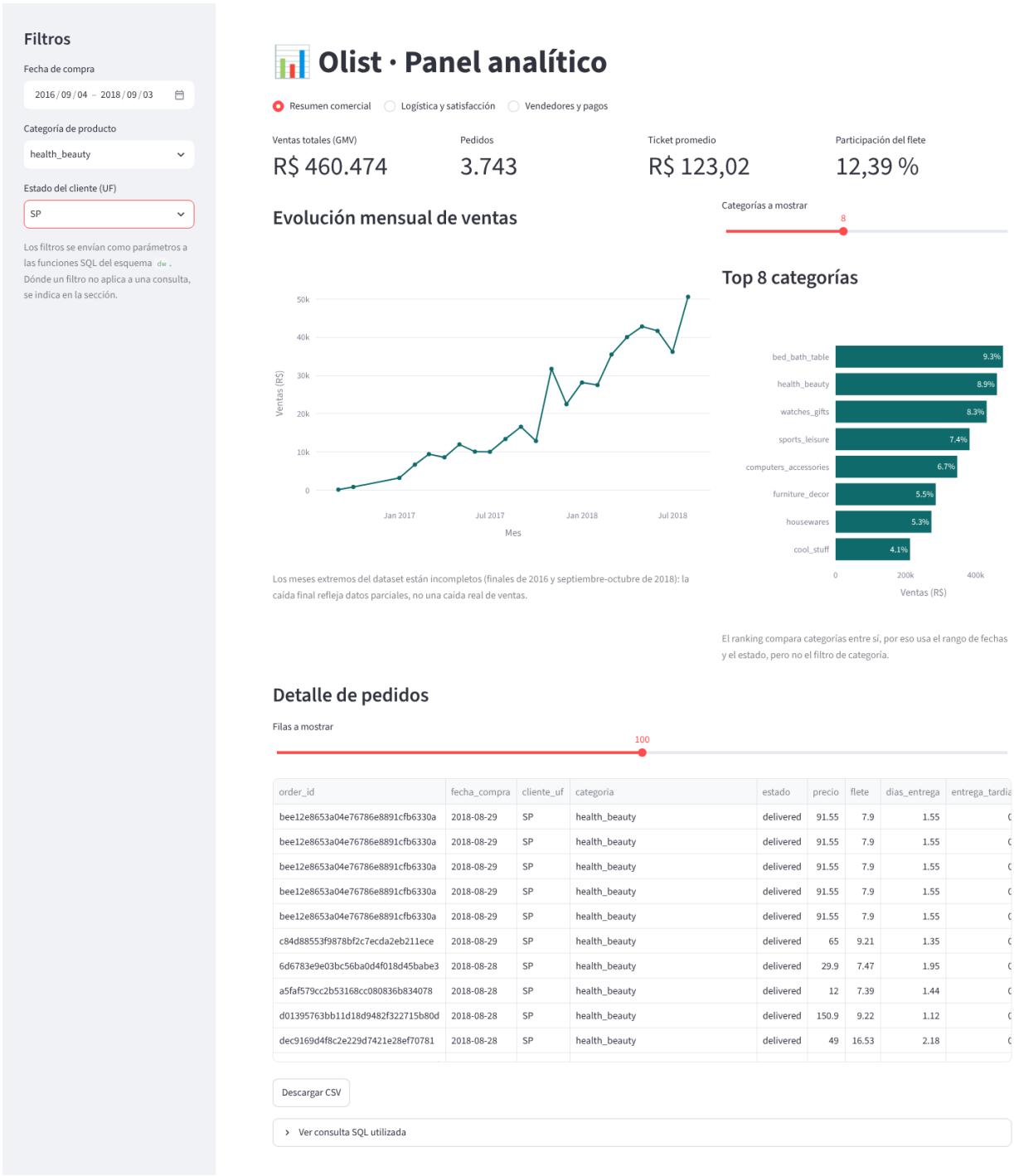



*Figura 4. Vista «Vendedores y pagos»: 13,20 % de ventas en el top 10, recompra de 3,04 % y medios de pago.*

## 17.4 Prueba de los filtros

Se cambiaron los filtros en la aplicación y se comprobó que los resultados cambian y coinciden con la ejecución directa de las funciones en PostgreSQL:

| **Filtros** | **GMV** | **Pedidos** | **Ticket** |
|---|---|---|---|
| Sin filtros | R$ 13.496.408 | 98.205 | R$ 137,43 |
| Categoría health_beauty | R$ 1.255.695 | 8.761 | R$ 143,33 |
| Categoría health_beauty + estado SP | R$ 460.474 | 3.743 | R$ 123,02 |
| 2018-01-01 a 2018-08-31 + health_beauty + SP | R$ 302.498 | — | R$ 123,47 |




| order_id | fecha_compra | cliente_uf | categoria | estado | precio | flete | dias_entrega | entrega_tardia |
|---|---|---|---|---|---|---|---|---|
| bee12e8653a04e76786e8891cfb6330a | 2018-08-29 | SP | health_beauty | delivered | 91.55 | 7.9 | 1.55 | 0 |
| bee12e8653a04e76786e8891cfb6330a | 2018-08-29 | SP | health_beauty | delivered | 91.55 | 7.9 | 1.55 | 0 |
| bee12e8653a04e76786e8891cfb6330a | 2018-08-29 | SP | health_beauty | delivered | 91.55 | 7.9 | 1.55 | 0 |
| bee12e8653a04e76786e8891cfb6330a | 2018-08-29 | SP | health_beauty | delivered | 91.55 | 7.9 | 1.55 | 0 |
| bee12e8653a04e76786e8891cfb6330a | 2018-08-29 | SP | health_beauty | delivered | 91.55 | 7.9 | 1.55 | 0 |
| c84d88553f9878bf2c7ecda2eb211ece | 2018-08-29 | SP | health_beauty | delivered | 65 | 9.21 | 1.35 | 0 |
| 6d6783e9e03bc56ba0d4f018d45babe3 | 2018-08-28 | SP | health_beauty | delivered | 29.9 | 7.47 | 1.95 | 0 |
| a5faf579cc2b53168cc080836b834078 | 2018-08-28 | SP | health_beauty | delivered | 12 | 7.39 | 1.44 | 0 |
| d01395763bb11d18d9482f322715b80d | 2018-08-28 | SP | health_beauty | delivered | 150.9 | 9.22 | 1.12 | 0 |
| dec9169d4f8c2e229d7421e28ef70781 | 2018-08-28 | SP | health_beauty | delivered | 49 | 16.53 | 2.18 | 0 |

*Figura 5. Vista «Resumen comercial» con categoría health_beauty y estado SP: el GMV baja a R$ 460.474 y cambian los gráficos y el detalle.*

## 17.5 Dónde se modifica cada cosa

| Si hay que… | Se edita |
|---|---|
| Cambiar o agregar un filtro | app/app.py (sección «filtros»: diccionario F) y app/db.py (parámetros ARGS) |
| Ver de dónde sale un KPI | Diccionario SQL de app/db.py → función dw.fn_* en sql/05_vistas_funciones.sql |
| Cambiar la conexión | get_engine() en app/db.py o variables de entorno PG* |
| Ejecutar una consulta a mano | docker compose exec db psql -U bi_user -d bi_database |

## 17.6 Resultados analíticos y decisiones que apoya

Cada hallazgo se obtuvo de la aplicación o de las funciones SQL de la sección 13, con el período completo.

| | | | |
|---|---|---|---|
| **R$ 13,5 M**<br>GMV (K1) en 98.205 pedidos no cancelados | **8,11 %**<br>entregas tardías (K5) sobre 12,56 días de entrega (K4) | **4,29 vs 2,57**<br>calificación a tiempo vs con retraso | **3,04 %**<br>tasa de recompra (K10) |

| Pregunta | Hallazgo | Decisión que podría apoyar |
|---|---|---|
| P1 | Las ventas crecen de 2017 (R$ 6,11 M) a 2018 (R$ 7,34 M, solo hasta agosto), con un pico en noviembre de 2017 (R$ 1,00 M). La región Sudeste concentra 65,4 % del GMV y solo São Paulo 38,3 %. Las categorías líderes son health_beauty (9,30 %), watches_gifts (8,88 %) y bed_bath_table (7,68 %). | Gerencia comercial: planificar campañas y stock para noviembre, priorizar las categorías líderes y evaluar cómo captar demanda en el Nordeste y el Norte (13,9 % del GMV entre ambos). |
| P2 | El 8,11 % de los pedidos llega tarde (7.827 de 96.476 con fecha). El problema es geográfico: AL (23,93 %), MA (19,67 %) y PI (15,97 %) están entre 2 y 3 veces por encima del promedio nacional, mientras SP tiene 5,89 % y 8,8 días de entrega frente a 24,5 días en AL. El flete equivale al 16,61 % de las ventas. | Logística: renegociar transportistas o ajustar los plazos estimados en el Nordeste y detectar rutas críticas; revisar el costo del flete como barrera de compra. |
| P3 | La puntualidad explica la satisfacción: los pedidos tardíos reciben 2,57 de calificación y 54,07 % de reseñas negativas; los puntuales, 4,29 y 9,22 %. | Servicio al cliente: contacto proactivo y compensación en pedidos con retraso; usar el % de tardías como alerta temprana de reseñas negativas. |
| P4 | La tarjeta de crédito concentra R$ 12,44 M (78 % del valor pagado) con un pago promedio de R$ 162,99, 3,51 cuotas en promedio y hasta 24. El boleto aporta R$ 2,85 M; el voucher tiene el pago más bajo (R$ 62,50). | Finanzas: definir políticas de financiamiento en cuotas y promociones por medio de pago. |
| P5 | Las ventas están poco concentradas: el top 10 de vendedores suma 13,20 %. Solo 3,04 % de los clientes repite compra. | Gestión de vendedores: riesgo de dependencia bajo; apoyar o advertir a vendedores del ranking con alto % de tardías o baja calificación. Comercial: programas de retención y recompra. |

### Limitaciones de la lectura

- Los meses extremos están incompletos (finales de 2016 y septiembre de 2018): la caída final del gráfico mensual no es una caída real.
- El dataset es de un solo marketplace y termina en 2018; las conclusiones describen ese período.
- La geografía se analiza por estado y región, sin coordenadas; el costo y los tiempos por ruta específica no se modelaron.

## 17.7 Limitaciones y trabajo futuro

- La geolocalización (1.000.163 filas) no entra en el modelo; un mapa por coordenadas sería una extensión natural.
- Una tercera tabla de hechos de reseñas (FACT_RESENAS, prevista en el Entregable 1) no fue necesaria: la calificación vive en el hecho de ventas, una vez por pedido.
- Las credenciales de la base son de laboratorio; en un despliegue real se manejarían con variables de entorno o un gestor de secretos.

# 18. README con instrucciones de ejecución

El archivo README.md del repositorio contiene las instrucciones completas de ejecución. Este es su resumen.

## 18.1 Requisitos

- Docker Desktop instalado y en ejecución.
- Los 9 archivos CSV del dataset (descargados de Kaggle).
- Python 3 con pip, para ejecutar la aplicación Streamlit.

## 18.2 Pasos para reproducir el proyecto

### Paso 1 · Preparar los datos

Descargar el dataset de Kaggle, descomprimir el zip y dejar los CSV directamente dentro de una carpeta archive/ en la raíz del proyecto (sin carpetas intermedias).

### Paso 2 · Levantar PostgreSQL

Desde la carpeta que contiene docker-compose.yml:

```
docker compose up -d
```

Conexión: host localhost, puerto 5432, base bi_database, usuario bi_user, contraseña bi_password. Si otro PostgreSQL usa el puerto 5432, cambiar en docker-compose.yml la línea "5432:5432" por "5433:5432" y conectarse a localhost:5433.

### Paso 3 · Ejecutar los scripts, en este orden

```
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/01_staging.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/02_modelo_dimensional.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/03_etl.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -v ON_ERROR_STOP=1 -f /data/sql/05_vistas_funciones.sql
docker compose exec -w /data/archive db psql -U bi_user -d bi_database -f /data/sql/04_validaciones.sql
```

El orden es 01, 02, 03, 05 y al final 04, porque las validaciones usan las funciones creadas en el script 05. Los scripts son re-ejecutables.

### Paso 4 · Comprobar que quedó bien

Resultados esperados de 04_validaciones.sql:

| Comprobación | Resultado esperado |
|---|---|
| Filas en dw.fact_ventas_item | 112.650 |
| Filas en dw.fact_pagos | 103.886 |
| Filas en dw.dim_cliente / dim_vendedor / dim_producto | 99.441 / 3.095 / 32.951 |
| Registros huérfanos en integridad referencial | 0 |
| Ventas totales (sin cancelados) | R$ 13.496.408,43 |
| Pedidos con al menos un ítem | 98.666 |

### Paso 5 · Ejecutar la aplicación Streamlit

```
python -m venv .venv
.venv\Scripts\activate          # Windows  (Linux/Mac: source .venv/bin/activate)
pip install -r app/requirements.txt
streamlit run app/app.py
```

Se abre en http://localhost:8501. La conexión usa por defecto los datos de docker-compose.yml; si se cambiaron, se definen las variables de entorno PGHOST, PGPORT, PGDATABASE, PGUSER y PGPASSWORD.

## 18.3 Comandos útiles de Docker

| Acción | Comando |
|---|---|
| Ver el estado del contenedor | docker compose ps |
| Detener (conserva los datos) | docker compose down |
| Borrar todo y empezar de cero | docker compose down -v |
| Abrir una consola SQL | docker compose exec db psql -U bi_user -d bi_database |

# 19. Evidencias de uso de Inteligencia Artificial

La IA generativa se usó como apoyo para proponer estructura y código. Todo el código se ejecutó sobre los datos reales y sus resultados se contrastaron con el origen; las decisiones de modelado y las correcciones indicadas fueron revisadas por el estudiante. Se documentan tres usos significativos.

### Uso 1 · Diseño y construcción del Data Mart

| | |
|---|---|
| **Problema** | Pasar de 9 CSV relacionados a un modelo dimensional en PostgreSQL, sin experiencia previa en SQL de carga (ETL). |
| **Qué se pidió** | Se entregó a la IA la consigna del Entregable 2, el informe del Entregable 1 y los CSV. |
| **Solución propuesta** | Dos capas (stg para datos crudos y dw para el Data Mart), tabla de hechos fact_ventas_item (una fila por ítem de pedido), seis dimensiones, una segunda tabla de hechos fact_pagos y scripts SQL 01 a 05. |
| **Qué se utilizó o modificó** | Se mantuvo el modelo en estrella del Entregable 1. Cambios propios o decididos tras revisar: dim_cliente a nivel de customer_id (porque cambia en cada pedido), la columna es_primer_item (evita contar un pedido varias veces) y la bitácora dw.log_calidad. |
| **Cómo se comprobó** | Con 04_validaciones.sql: mismas filas en origen y destino (112.650 ítems), 0 huérfanos, 0 duplicados en la granularidad y totales idénticos al centavo (GMV R$ 13.496.408,43 en stg y en dw). Los scripts se ejecutaron de cero sin errores. |

### Uso 2 · Cifras que no coincidían con el Entregable 1

| | |
|---|---|
| **Problema** | El tiempo medio de entrega del Entregable 1 (12,1 días) no coincidía con el calculado en el Data Mart (12,56 días). |
| **Qué se pidió** | Validar las cifras del Entregable 1 contra los datos cargados en PostgreSQL. |
| **Solución propuesta** | La IA contrastó definiciones: el 12,1 salía de truncar cada entrega a días enteros (promedio 12,09), mientras que el modelo usa fracciones de día. Las 7.827 entregas tardías sí coincidían al comparar fecha y hora. |
| **Qué se utilizó o modificó** | Se adoptó el cálculo con fracciones de día y se documentó la diferencia. También se explicó que calificación, % de reseñas negativas y recompra cambian levemente porque ahora se calculan por pedido y sin cancelados. |
| **Cómo se comprobó** | Consulta directa sobre stg.orders con ambas definiciones (12,09 truncando; 12,56 con fracciones) y conteo de tardías (7.827 de 96.476). |

### Uso 3 · Aplicación Streamlit conectada a PostgreSQL

| | |
|---|---|
| **Problema** | Construir un dashboard con filtros que afecten de verdad a las consultas, sin leer nunca los CSV. |
| **Qué se pidió** | Se entregó a la IA la consigna del Entregable 3 y se pidió una aplicación Streamlit sobre las funciones SQL del Entregable 2. |
| **Solución propuesta** | db.py (conexión con SQLAlchemy, consultas parametrizadas y caché) y app.py (3 vistas, 3 filtros, KPI, gráficos con Plotly y tabla de detalle). Los filtros viajan como parámetros a las funciones SQL. |
| **Qué se utilizó o modificó** | Se corrigió una nota errónea de la IA que decía que el ranking de categorías respondía al filtro de categoría (la función SQL no lo recibe); ahora la aplicación indica qué filtro no aplica en cada consulta. Se agregó una nota sobre los meses incompletos, para que la caída final del gráfico no se interprete como caída real de ventas. |
| **Cómo se comprobó** | Con los filtros por defecto los KPI coinciden con 04_validaciones.sql; al elegir health_beauty el GMV baja de R$ 13.496.408 a R$ 1.255.695 y, con el estado SP, a R$ 460.474 (sección 17.4). Se revisó que la aplicación no importa ni abre ningún CSV de origen. |

# 20. Video individual de 3 a 5 minutos

El video individual dura entre 3 y 5 minutos y se entrega junto con este documento. Muestra evidencia real del funcionamiento del proyecto (no solo diapositivas) y explica un problema en el que la IA ayudó, qué se adaptó o validó.

Enlace del video: ENTREGABLES

## 20.1 Estructura del video

| Tiempo | Qué se muestra | Qué se explica |
|---|---|---|
| 0:00 – 0:30 | Portada o README | Qué se analizó: pedidos de Olist de 2016 a 2018 en 9 tablas, y las decisiones que apoya (categorías y regiones, retrasos, experiencia del cliente). |
| 0:30 – 1:15 | Diagrama estrella | Granularidad: una fila de fact_ventas_item es un ítem de un pedido; seis dimensiones y una segunda tabla de hechos para pagos. |
| 1:15 – 2:00 | pgAdmin | Esquema dw con los datos cargados, SELECT count(*) |

| | | |
|---|---|---|
| | | FROM dw.fact_ventas_item (112.650) y una función como dw.fn_total_ventas(). |
| 2:00 – 3:30 | Aplicación Streamlit | Recorrido por las tres vistas; cambio de un filtro (health_beauty y SP) y cómo cambian el GMV y los gráficos; «Ver consulta SQL utilizada». |
| 3:30 – 4:10 | Vista «Logística y satisfacción» | Un resultado: los pedidos con retraso reciben 2,57 de calificación frente a 4,29 de los puntuales. |
| 4:10 – 4:40 | Evidencias de IA | Caso del tiempo de entrega (12,1 contra 12,56 días): qué propuso la IA, qué se validó contra los datos y qué se adaptó. |